Lab 2: Netconf, Mininet, OpenFlow, OpenVSwitch, Controllers (ODL, POX, ONOS, Ryu, . . .)

Anais Sadoudi
M2 CNS - SR - SDN
Université d'Évry, Paris-Saclay

October 2026

Contents

1 Introduction **3**

2 Exercise 1: Mininet **3**
2.1 Question 1 . . . 3
2.2 Question 2 . . . 5
2.3 Question 3 . . . 8

3 Exercise 2: Breaking a network **10**
3.1 without STP . . . 10
3.2 with STP . . . 11

4 Exercise 3: Understanding OpenFlow messages between a SDN controller and an OF Switch **14**
4.1 Setup . . . 14
4.2 Quick start . . . 14
4.3 Understanding OpenFlow messages . . . 15

5 Exercise 4: dpctl and ovs-dpctl **20**
5.1 Differences with the lab statement . . . 20
5.2 Setup . . . 21
5.3 Question 1 . . . 22
5.4 Question 2 . . . 23
5.5 Question 3 . . . 24

6 Exercise 5: Topology discovery **25**
6.1 Setup (OpenDaylight) . . . 25
6.2 Question 1 . . . 25
6.3 Question 2 . . . 28
6.4 Question 3 . . . 29
6.5 Question 4 . . . 30
6.6 Question 5 . . . 32
6.7 Question 6 . . . 32
6.8 Setup (ONOS) . . . 33
6.9 Question 2 (ONOS) . . . 34
6.10 Question 3 (ONOS) . . . 35
6.11 Question 4 (ONOS) . . . 36

6.12 Question 5 (ONOS) 38
6.13 Question 6 (ONOS) 38

7 Exercise 6: Switching rules **38**
7.1 Question 1 39
7.2 Question 3 39
7.3 Question 4 39
7.4 Question 6 39
7.5 Question 7 39
7.6 Question 8 40

1 Introduction

Software Defined Networking (SDN) divides the network into a data plane that does the actual packet forwarding and the control plane that makes the decisions on how to forward the data. With SDN, switches are just dumb forwarders that can be programmed to behave a certain way with a standard protocol by a logically centralised controller. OpenFlow [**?**, **?**].

2 Exercise 1: Mininet

2.1 Question 1

Write a Python program to use the high-level API and create a simple-tree data center topology. The program gets the value of n as an input. You must organize all the source code in a single source file, name it simpletree-highlevel.py, and include this file in the submission archive.

The high-level API is based on the Topo class. I define a class SimpleTree that inherits from Topo and overrides its build() method. Inside build(), three nested loops create the aggregation, edge and host layers with addSwitch(), addHost() and addLink(). At this stage, the class only describes the topology.

```
mininet@mininet-vm:~/lab2$ cat simpletree_highlevel.py
import sys
from mininet.topo import Topo
from mininet.net import Mininet
from mininet.cli import CLI
from mininet.log import setLogLevel


class SimpleTree(Topo):

    def build(self, n=2):
        self.dpid = 1
        core = self.newSwitch('C1')

        h = 1
        e = 1

        for a in range(1, n + 1):
            aggregation = self.newSwitch('A%d' % a)
            self.addLink(core, aggregation)

            for _ in range(n):
                edge = self.newSwitch('E%d' % e)
                e += 1
                self.addLink(aggregation, edge)

                for _ in range(n):
                    host = self.addHost('H%d' % h)
                    h += 1
                    self.addLink(edge, host)
    def newSwitch(self, name):
       sw = self.addSwitch(name, dpid='%016x' % self.dpid)
       self.dpid += 1
       return sw

if __name__ == '__main__':
    setLogLevel('info')
    n = int(sys.argv[1]) if len(sys.argv) > 1 else int(input('n = '))
    net = Mininet(topo=SimpleTree(n=n))
    net.start()
    CLI(net)
    net.stop()
mininet@mininet-vm:~/lab2$
```

Figure 1: Script of simpletree highlevel.py.

```
mininet@mininet-vm:~/lab2$ sudo python3 simpletree_highlevel.py 2
*** Creating network
*** Adding controller
*** Adding hosts:
H1 H2 H3 H4 H5 H6 H7 H8
*** Adding switches:
A1 A2 C1 E1 E2 E3 E4
*** Adding links:
(A1, E1) (A1, E2) (A2, E3) (A2, E4) (C1, A1) (C1, A2) (E1, H1) (E1, H2) (E2, H3) (E2, H4) (E3, H5) (E3, H6) (E4, H7) (E4, H8)
*** Configuring hosts
H1 H2 H3 H4 H5 H6 H7 H8
*** Starting controller
c0
*** Starting 7 switches
A1 A2 C1 E1 E2 E3 E4 ...
*** Starting CLI:
mininet> nodes
available nodes are:
A1 A2 C1 E1 E2 E3 E4 H1 H2 H3 H4 H5 H6 H7 H8 c0
mininet> net
H1 H1-eth0:E1-eth2
H2 H2-eth0:E1-eth3
H3 H3-eth0:E2-eth2
H4 H4-eth0:E2-eth3
H5 H5-eth0:E3-eth2
H6 H6-eth0:E3-eth3
H7 H7-eth0:E4-eth2
H8 H8-eth0:E4-eth3
A1 lo:  A1-eth1:C1-eth1 A1-eth2:E1-eth1 A1-eth3:E2-eth1
A2 lo:  A2-eth1:C1-eth2 A2-eth2:E3-eth1 A2-eth3:E4-eth1
C1 lo:  C1-eth1:A1-eth1 C1-eth2:A2-eth1
E1 lo:  E1-eth1:A1-eth2 E1-eth2:H1-eth0 E1-eth3:H2-eth0
E2 lo:  E2-eth1:A1-eth3 E2-eth2:H3-eth0 E2-eth3:H4-eth0
E3 lo:  E3-eth1:A2-eth2 E3-eth2:H5-eth0 E3-eth3:H6-eth0
E4 lo:  E4-eth1:A2-eth3 E4-eth2:H7-eth0 E4-eth3:H8-eth0
c0
mininet> exit
*** Stopping 1 controllers
c0
*** Stopping 14 links
..............
*** Stopping 7 switches
A1 A2 C1 E1 E2 E3 E4
*** Stopping 8 hosts
```

Figure 2: Test of the script.

The net output confirms the expected structure: C1 is connected to A1 and A2, A1 to E1 and E2, A2 to E3 and E4, and each edge switch to two hosts.

2.2 Question 2

Repeat the previous question but now use the low-level API. You must organize all the source code in a single source file, name it simpletree-lowlevel.py, and include this file in the submission archive.

The low-level API, as described in [1], uses the node and link classes directly (Host, OVSSwitch, DefaultController, Link) without any Topo or Mininet object. The same three nested loops are used, but each object is created immediately.

```
mininet@mininet-vm:~/lab2$ cat simpletree_lowlevel.py
import sys
from mininet.node import Host, OVSSwitch, DefaultController
from mininet.link import Link
from mininet.log import setLogLevel, info

def build(n):
    switches = []
    hosts = []
    dpid = [1]

    C0 = DefaultController('C0', inNamespace=False)

    def newSwitch(name):
        sw = OVSSwitch(name, dpid='%016x' % dpid[0], inNamespace=False)
        dpid[0] += 1
        switches.append(sw)
        return sw
    core = newSwitch('C1')
    h = 1
    e = 1

    for a in range(1, n + 1):
        agg = newSwitch('A%d' % a)
        Link(core, agg)

        for _ in range(n):
            edge = newSwitch('E%d' % e)
            e += 1
            Link(agg, edge)

            for _ in range(n):
                host = Host('H%d' % h)
                Link(edge, host)
                host.setIP('10.0.0.%d/8' % h)
                hosts.append(host)
                h += 1

    return C0, switches, hosts

def pingAll(hosts):
    sent = 0
    ok = 0
    for src in hosts:
```

Figure 3: Script of simpletree-lowlevel.py.

```
def pingAll(hosts):
    sent = 0
    ok = 0
    for src in hosts:
        result = src.name + '->'
        for dst in hosts:
            if src == dst:
                continue
            out = src.cmd('ping -c1 -W2 %s' % dst.IP())
            sent += 1
            if '1 received' in out:
                ok += 1
                result += dst.name + ' '
            else:
                result += 'X '
        info(result + '\n')
    info('*** Results: %d/%d received\n' % (ok, sent))


if __name__ == '__main__':
    setLogLevel('info')
    n = int(sys.argv[1]) if len(sys.argv) > 1 else int(input('n = '))

    info('*** Creating nodes and links\n')
    C0, switches, hosts = build(n)

    info('*** Starting controller and switches\n')
    C0.start()
    for sw in switches:
        sw.start([C0])

    info('*** Testing connectivity\n')
    pingAll(hosts)

    input('*** Press Enter to stop...')

    info('*** Stopping\n')
    for sw in switches:
        sw.stop()
    for host in hosts:
        host.terminate()
    C0.stop()
mininet@mininet-vm:~/lab2$
```

Figure 4: Script of simpletree-lowlevel.py.

```
mininet@mininet-vm:~/lab2$ sudo python3 simpletree_lowlevel.py 2
*** Creating nodes and links
*** Starting controller and switches
*** Testing connectivity
H1 -> H2 H3 H4 H5 H6 H7 H8
H2 -> H1 H3 H4 H5 H6 H7 H8
H3 -> H1 H2 H4 H5 H6 H7 H8
H4 -> H1 H2 H3 H5 H6 H7 H8
H5 -> H1 H2 H3 H4 H6 H7 H8
H6 -> H1 H2 H3 H4 H5 H7 H8
H7 -> H1 H2 H3 H4 H5 H6 H8
H8 -> H1 H2 H3 H4 H5 H6 H7
*** Results: 56/56 received
*** Press Enter to stop...
*** Stopping
```

Figure 5: Test of the script of simpletree-lowlevel.py.

2.3 Question 3

Use your program (either high-level or low-level) to create a simple-tree with n=2, as shown in previous Figure. Check the connectivity of all nodes in the network and write your observations. Now stop the core switch using command switch c1 stop, check the connectivity of all hosts, and report your observations. How do you explain your findings? Write up your answer in the report.

The high-level script was run with n = 2. A first pingall gives 0% packet loss.

```
mininet@mininet-vm:~/lab2$ sudo python3 simpletree_highlevel.py 2
*** Creating network
*** Adding controller
*** Adding hosts:
H1 H2 H3 H4 H5 H6 H7 H8
*** Adding switches:
A1 A2 C1 E1 E2 E3 E4
*** Adding links:
(A1, E1) (A1, E2) (A2, E3) (A2, E4) (C1, A1) (C1, A2) (E1, H1) (E1, H2) (E2, H3) (E2, H4) (E3, H5) (E3, H6) (E4, H7) (E4, H8)
*** Configuring hosts
H1 H2 H3 H4 H5 H6 H7 H8
*** Starting controller
c0
*** Starting 7 switches
A1 A2 C1 E1 E2 E3 E4 ...
*** Starting CLI:
mininet> pingall
*** Ping: testing ping reachability
H1 -> H2 H3 H4 H5 H6 H7 H8
H2 -> H1 H3 H4 H5 H6 H7 H8
H3 -> H1 H2 H4 H5 H6 H7 H8
H4 -> H1 H2 H3 H5 H6 H7 H8
H5 -> H1 H2 H3 H4 H6 H7 H8
H6 -> H1 H2 H3 H4 H5 H7 H8
H7 -> H1 H2 H3 H4 H5 H6 H8
H8 -> H1 H2 H3 H4 H5 H6 H7
*** Results: 0% dropped (56/56 received)
```

Figure 6: ...

The core switch is stopped with switch C1 stop, and pingall is run again.

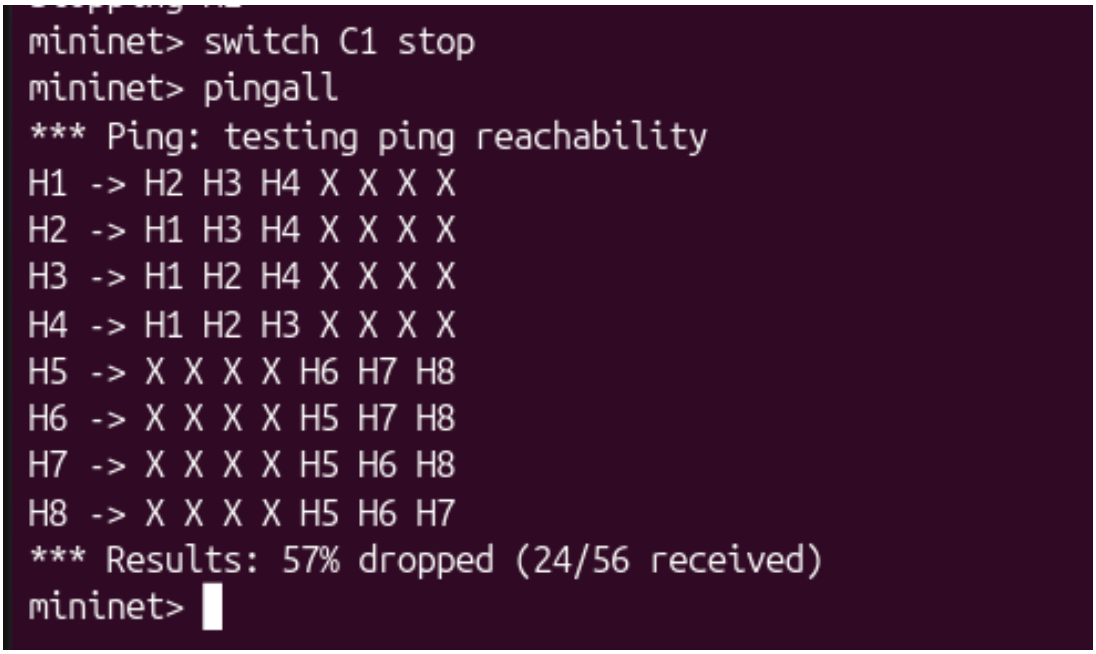


Figure 7: ...

After C1 is stopped, only 24 of the 56 pings succeed. Hosts H1–H4 can still reach each other, and so can H5–H8, but no host in one half can reach a host in the other half.

I restart C1 with switch C1 start restores full connectivity.

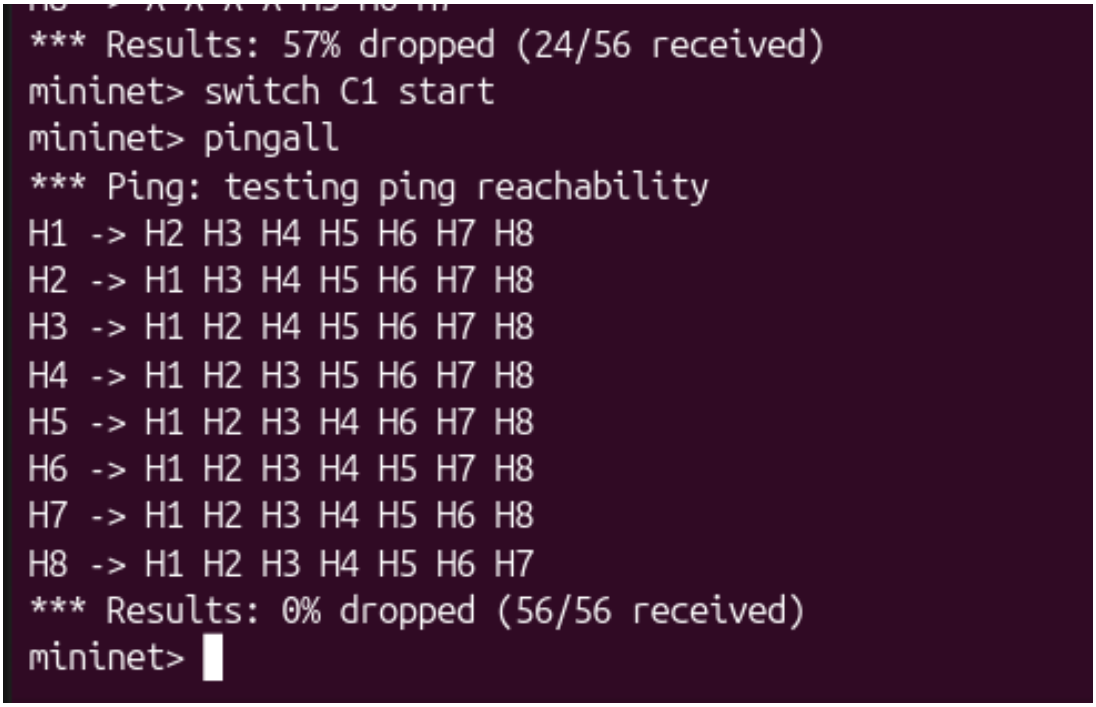


Figure 8: ...

In a tree topology there is exactly one path between any two nodes. Any traffic between the two subtrees must go through the core switch C1, which is therefore a single point of failure: when it fails, the network is partitioned into two isolated parts.

3 Exercise 2: Breaking a network

Three switches are connected in a triangle (s1–s2–s3), with one host attached to each switch. The link s3–s1 closes the loop: there are two paths between any pair of switches.

3.1 without STP

After starting the network, h1 ping -c3 h2 fails with 100% packet loss and the message Destination Host Unreachable, before sending an ICMP packet, h1 must resolve the MAC address of h2 with an ARP request. This resolution never completed, so no ICMP packet is actually sent.

```
mininet@mininet-vm:~/lab2$ sudo python3 loop_topology.py
*** Adding switches (no STP)
*** Adding hosts
*** Creating links
*** Configuring hosts
h1 h2 h3
*** Starting controller

*** Starting 3 switches
s1 s2 s3 ...
*** Starting CLI:
mininet> h1 ping -c3 h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.786 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.119 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.074 ms

--- 10.0.0.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2006ms
rtt min/avg/max/mdev = 0.074/0.326/0.786/0.325 ms
mininet> h1 ping -c3 h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.364 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.086 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.089 ms

--- 10.0.0.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2040ms
rtt min/avg/max/mdev = 0.086/0.179/0.364/0.130 ms
mininet> exit
```

Figure 9: Ping from h1 to h2 in the loop topology without STP.

The ARP request is a broadcast frame, so each switch floods it on all its ports except the incoming one. Because of the loop, every copy comes back to the switches and is flooded again, in both directions, and the number of frames grows without limit. Unlike IP packets, Ethernet frames have no TTL field, so nothing ever removes them from the network. It's a `broadcast storm`.

3.2 with STP

To confirm that the loop is the cause, I start the same topology with stp=True on the three switches. A first ping, sent right after start-up, still failed: STP ports go through the listening and learning states before forwarding data. After some s, the ping succeeds with 0% loss.

ovs-appctl stp/show shows the resulting spanning tree:

s1 is elected root bridge: all priorities are equal (32768), so the bridge with the lowest MAC address wins. All its ports are designated and forwarding.

s2 and s3 both reach the root through their direct link to s1

On the redundant segment s2–s3, both bridges have the same root path cost, so the tie is broken by the bridge ID: s3 (a2:66:...) is lower than s2 (ae:f3:...), so s3 keeps the designated port and port s2-eth3 is put in the alternate/blocking state.

The physical loop still exists, but the active topology is a tree (s1–s2, s1–s3), so no frame can circulate endlessly.

```
mininet> sh ovs-vsctl get bridge s1 stp_enable
true
mininet> sh ovs-appctl stp/show s1
---- s1 ----
Root ID:
  stp-priority  32768
  stp-system-id   1e:8a:99:b8:77:46
  stp-hello-time  2s
  stp-max-age     20s
  stp-fwd-delay   15s
  This bridge is the root
```

Figure 10: stp/show s1.

```
mininet> sh ovs-appctl stp/show s2
---- s2 ----
Root ID:
  stp-priority  32768
  stp-system-id   1e:8a:99:b8:77:46
  stp-hello-time  2s
  stp-max-age     20s
  stp-fwd-delay   15s
  root-port       s2-eth2
  root-path-cost  2

Bridge ID:
  stp-priority  32768
  stp-system-id   ae:f3:be:55:9b:40
  stp-hello-time  2s
  stp-max-age     20s
  stp-fwd-delay   15s

  Interface  Role       State      Cost  Pri.Nbr
  ---------- ---------- ---------- ----- -------
  s2-eth3    alternate  blocking   2     128.1
  s2-eth2    root       forwarding 2     128.2
  s2-eth1    designated forwarding 2     128.3

mininet> sh ovs-appctl stp/show s3
---- s3 ----
Root ID:
  stp-priority  32768
```

Figure 11: stp/show s2.

```
mininet> sh ovs-appctl stp/show s3
---- s3 ----
Root ID:
  stp-priority  32768
  stp-system-id   1e:8a:99:b8:77:46
  stp-hello-time  2s
  stp-max-age     20s
  stp-fwd-delay   15s
  root-port       s3-eth3
  root-path-cost  2

Bridge ID:
  stp-priority  32768
  stp-system-id   a2:66:41:33:b3:4b
  stp-hello-time  2s
  stp-max-age     20s
  stp-fwd-delay   15s

  Interface  Role       State      Cost  Pri.Nbr
  ---------- ---------- ---------- ----- -------
  s3-eth1    designated forwarding 2     128.1
  s3-eth3    root       forwarding 2     128.2
  s3-eth2    designated forwarding 2     128.3
```

Figure 12: stp/show s3.

```
mininet> h1 ping -c3 h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.647 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.091 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.083 ms

--- 10.0.0.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2026ms
rtt min/avg/max/mdev = 0.083/0.273/0.647/0.264 ms
mininet>
```

Figure 13: Ping from h1 to h2 with STP

A loop in a layer-2 network without a loop-prevention mechanism leads to a broadcast storm that makes the network unusable. STP solves this problem in a distributed way by logically blocking redundant links, at the cost of a convergence delay.

4 Exercise 3: Understanding OpenFlow messages between a SDN controller and an OF Switch

4.1 Setup

The lab statement suggests installing python2-ryu. Since the Mininet VM runs Ubuntu 20.04 with Python 3.8, I install Ryu 4.34 for Python 3 with pip3 install ryu. The eventlet library was downgraded to version 0.30.2, because recent versions removed functions used by Ryu 4.34.

```
mininet@mininet-vm:~$ ryu-manager --version
ryu-manager 4.34
mininet@mininet-vm:~$ ryu-manager ryu.app.simple_switch_13
*loading app ryu.app.simple_switch_13
loading app ryu.controller.ofp_handler
instantiating app ryu.app.simple_switch_13 of SimpleSwitch13
instantiating app ryu.controller.ofp_handler of OFPHandler
```

Figure 14: ryu-manager version.

4.2 Quick start

I start Mininet with one OVS switch and three hosts, restricted to OpenFlow 1.3 and connected to a remote controller:

```
sudo mn --topo single,3 --mac --controller remote --switch ovsk,protocols=OpenFlow13
```

Ryu is started with the `simple_switch_13` application, which implements a MAC-learning switch for OpenFlow 1.3. The OpenFlow traffic is captured on the loopback interface with `tcpdump`, since the controller and the switch run on the same machine and communicate through 127.0.0.1. The Mininet VM has no graphical interface, so the capture file was analysed with Wireshark on the host computer.

```
mininet@mininet-vm:~$ ryu-manager --verbose ryu.app.simple_switch_13
loading app ryu.app.simple_switch_13
loading app ryu.controller.ofp_handler
instantiating app ryu.app.simple_switch_13 of SimpleSwitch13
instantiating app ryu.controller.ofp_handler of OFPHandler
BRICK SimpleSwitch13
  CONSUMES EventOFPPacketIn
  CONSUMES EventOFPSwitchFeatures
BRICK ofp_event
  PROVIDES EventOFPPacketIn TO {'SimpleSwitch13': {'main'}}
  PROVIDES EventOFPSwitchFeatures TO {'SimpleSwitch13': {'config'}}
  CONSUMES EventOFPEchoReply
  CONSUMES EventOFPEchoRequest
  CONSUMES EventOFPErrorMsg
  CONSUMES EventOFPHello
  CONSUMES EventOFPPortDescStatsReply
  CONSUMES EventOFPPortStatus
  CONSUMES EventOFPSwitchFeatures
connected socket:<eventlet.greenio.base.GreenSocket object at 0x7f4b0b627c10> address:('127.0.0.1', 39612)
connected socket:<eventlet.greenio.base.GreenSocket object at 0x7f4b0b627d90> address:('127.0.0.1', 39614)
hello ev <ryu.controller.ofp_event.EventOFPHello object at 0x7f4b0b699850>
move onto config mode
EVENT ofp_event->SimpleSwitch13 EventOFPSwitchFeatures
switch features ev version=0x4,msg_type=0x6,msg_len=0x20,xid=0xd9b6d08,OFPSwitchFeatures(auxiliary_id=0,capabilities=79,datapath_id=1,n_buffers=0,n_tables=254)
move onto main mode
```

Figure 15: Ryu `simple_switch_13` started in verbose mode: application components (bricks) and OpenFlow handshake with switch s1 (HELLO, FEATURES_REPLY).

```
mininet@mininet-vm:~$ sudo mn --topo single,3 --mac --controller remote --switch ovsk,protocols=OpenFlow13
*** Creating network
*** Adding controller
Connecting to remote controller at 127.0.0.1:6653
*** Adding hosts:
h1 h2 h3
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1) (h3, s1)
*** Configuring hosts
h1 h2 h3
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet> h1 ping -c3 h3
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
64 bytes from 10.0.0.3: icmp_seq=1 ttl=64 time=22.1 ms
64 bytes from 10.0.0.3: icmp_seq=2 ttl=64 time=0.302 ms
64 bytes from 10.0.0.3: icmp_seq=3 ttl=64 time=0.363 ms

--- 10.0.0.3 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2032ms
rtt min/avg/max/mdev = 0.302/7.575/22.062/10.243 ms
mininet>
```

Figure 16: `h1 ping h3` through the Ryu controller: the first packet has a much higher RTT than the following ones.

```
move onto main mode
EVENT ofp_event->SimpleSwitch13 EventOFPPacketIn
packet in 1 00:00:00:00:00:01 ff:ff:ff:ff:ff:ff 1
EVENT ofp_event->SimpleSwitch13 EventOFPPacketIn
packet in 1 00:00:00:00:00:03 00:00:00:00:00:01 3
EVENT ofp_event->SimpleSwitch13 EventOFPPacketIn
packet in 1 00:00:00:00:00:01 00:00:00:00:00:03 1
```

Figure 17: PACKET_IN events received by Ryu during the ping (ARP request, ARP reply and first ICMP echo request).

Only three PACKET_IN messages are received by the controller

```
mininet> sh ovs-ofctl dump-flows s1 -O OpenFlow13
 cookie=0x0, duration=88.397s, table=0, n_packets=4, n_bytes=336, priority=1,in_port="s1-eth3",dl_src=00:00:00:00:00:03,dl_dst=00:00:00:00:00:01 actions=output:"s1-eth1"
 cookie=0x0, duration=88.389s, table=0, n_packets=3, n_bytes=238, priority=1,in_port="s1-eth1",dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:03 actions=output:"s1-eth3"
 cookie=0x0, duration=139.030s, table=0, n_packets=3, n_bytes=182, priority=0 actions=CONTROLLER:65535
mininet>
```

Figure 18: Flow table of s1 (`ovs-ofctl dump-flows s1 -O OpenFlow13`) after the ping: table-miss entry and two rules installed by Ryu.

4.3 Understanding OpenFlow messages

The capture is open in Wireshark and filter with `openflow_v4`. Two TCP connections reach the controller: the first one (port 39612) is a short connection opened by Mininet to check that a controller is listening, and only receives a HELLO from Ryu. The OpenFlow session with switch s1 is the second connection (source port 39614), which was isolated with the filter `tcp.stream eq 1 && openflow_v4`.

Connection setup (OFPT_HELLO)

The switch opens a TCP connection to the controller on port 6653, the standard OpenFlow port. Each side then sends an OFPT_HELLO message whose version field is set to the highest OpenFlow version it supports[1]. Both messages carry version 0x04, i.e. OpenFlow 1.3. The HELLO sent by the switch is 16 bytes long: in addition to the 8-byte header, it contains a version bitmap element listing exactly the versions it supports [1], since the switch was restricted with `protocols=OpenFlow13`, only OpenFlow 1.3 is set. The HELLO sent by Ryu (Figure 20) only contains the 8-byte header. The negotiated version is the highest version supported by both sides[1], so OpenFlow 1.3 is used for all subsequent messages.

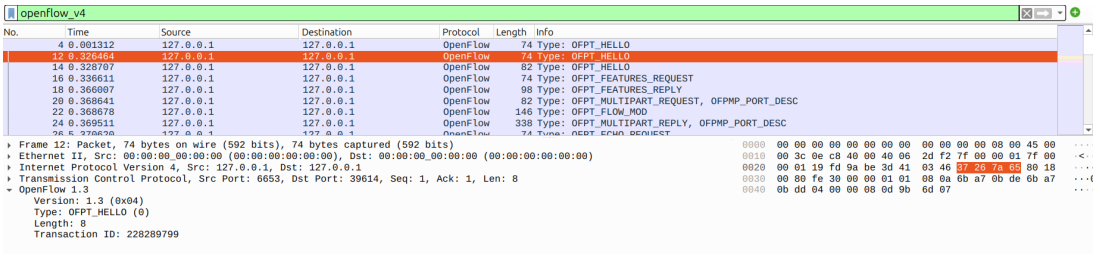


Figure 19: OFPT_HELLO sent by the switch to the controller (version 0x04, with version bitmap).

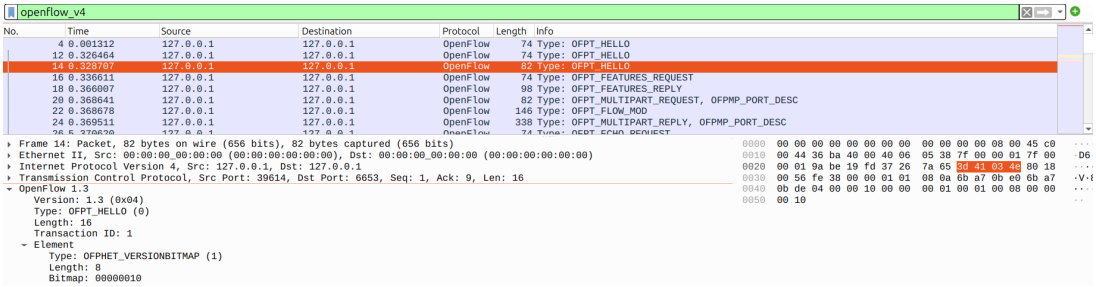


Figure 20: OFPT_HELLO sent by Ryu to the switch (version 0x04, header only).

Feature request / reply

Once the session is established, the controller sends an OFPT_FEATURES_REQUEST (type 5. This message only contains the 8-byte OpenFlow header and no body: it simply asks the switch to identify itself. The switch answers with an OFPT_FEATURES_REPLY (type 6, carrying the same transaction ID (228289800), which allows the controller to match the reply to its request.

The FEATURES_REPLY contains:

- **Datapath ID** = 0x0000000000000001: the unique 64-bit identifier of the switch, used by the controller to refer to it;
- **Buffers** = 0: the switch does not buffer packets sent to the controller, so whole packets are transmitted in PACKET_IN messages;
- **Tables** = 254: number of flow tables available in the OpenFlow pipeline;
- **Auxiliary ID** = 0: this is the main connection;
- **Capabilities** = 0x0000004f: flow, table, port, group and queue statistics are supported, while IP reassembly and blocking of looping ports are not.

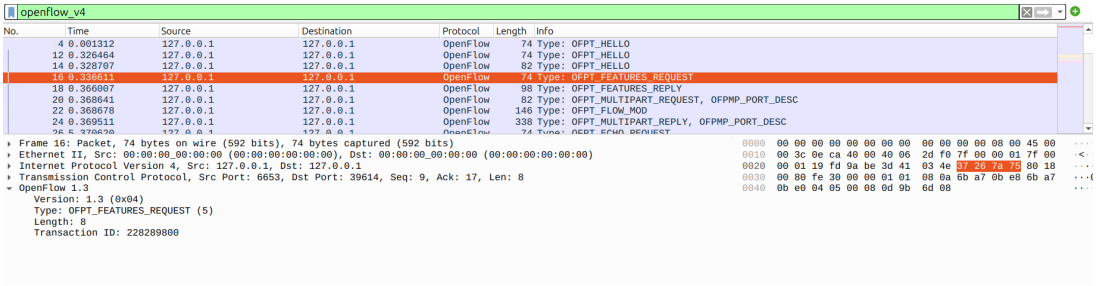


Figure 21: OFPT_FEATURES_REQUEST sent by Ryu .

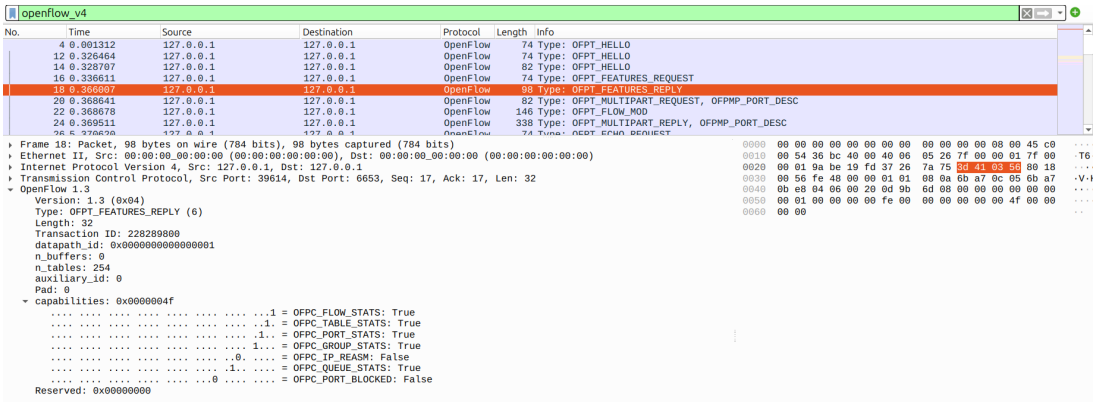


Figure 22: OFPT_FEATURES_REPLY sent by the switch: datapath ID, buffers, tables and capabilities.

Set configuration

According to the OpenFlow 1.3 specification[1], the controller may send an OFPT_SET_CONFIG message (type 9) to set two global parameters of the switch: the **flags**, which define how IP fragments are handled and whether packets with an invalid TTL are sent to the controller, and **miss_send_len**, the maximum number of bytes of a packet sent to the controller in a PACKET_IN when no other limit is specified. In my capture, no SET_CONFIG message was found, with Ryu 4.34, the controller goes directly from the FEATURES_REPLY to the port description request, and the switch keeps its default configuration.

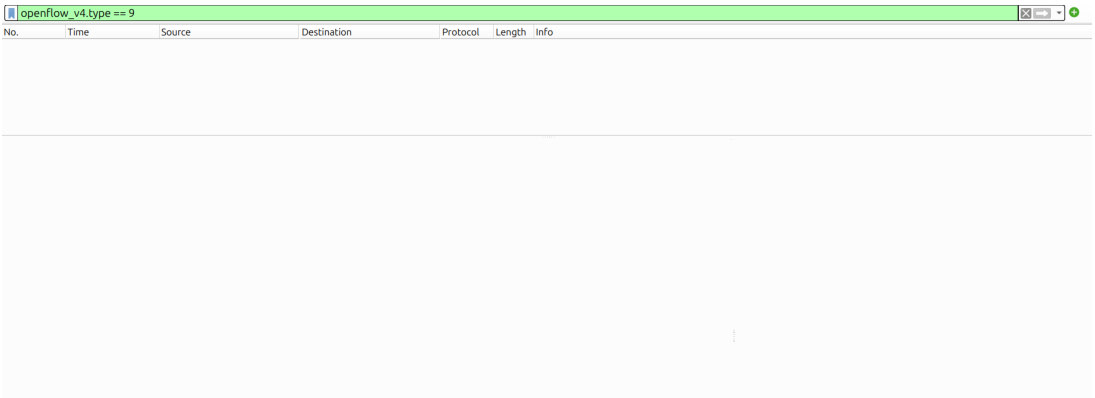


Figure 23: No OFPT_SET_CONFIG message in the capture (filter `openflow_v4.type == 9`).

Multipart request / reply

OFPT_MULTIPART_REQUEST (type 18) is the generic message used by the controller to read the state of the switch[1]. It replaces the STATS messages of OpenFlow 1.0 and supports many sub-types, either statistics (FLOW, TABLE, PORT, QUEUE, METER, ...) or descriptions (DESC, PORT_DESC, TABLE_FEATURES, ...).

Right after the FEATURES_REPLY, Ryu sends a MULTIPART_REQUEST of type OFPMP_PORT_DESC (13) with no flag set. The switch answers with an OFPT_MULTIPART_REPLY (type 19) carrying the same transaction ID. Its body contains one 64-byte description per port[1], four ports connected to the three hosts, and the internal port `s1` (OFPP_LOCAL, 0xfffffffe). Each description gives the port number, its MAC address, its name, its configuration and state, and its current and supported features and speeds.

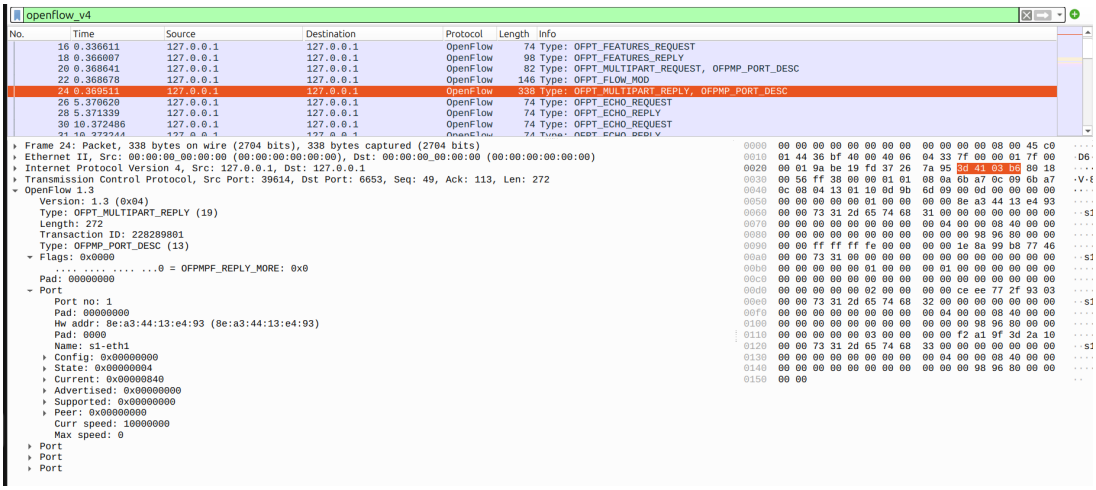


Figure 24: OFPT_MULTIPART_REQUEST of type OFPMP_PORT_DESC sent by Ryu.

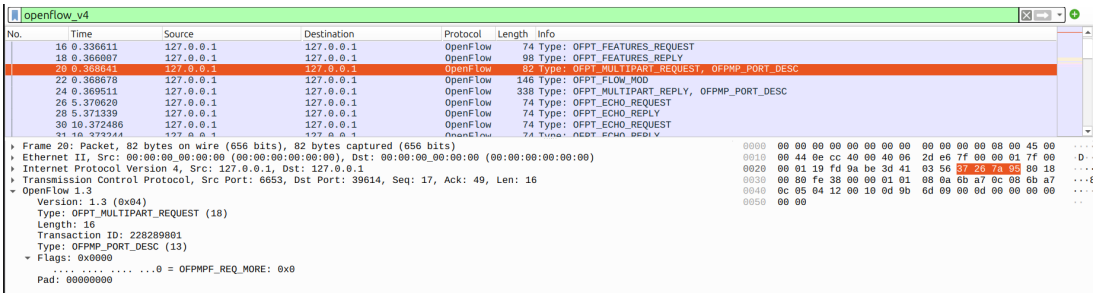


Figure 25: OFPT_MULTIPART_REPLY (PORT_DESC) sent by the switch: description of its ports.

Flow mod

OFPT_FLOW_MOD is used by the controller to add, modify or delete flow entries [1]. My capture contains three of them. The first one (packet 22) is sent proactively when the switch connects and installs the table-miss entry. The two others (61 and 66) are sent reactively after PACKET_IN messages

The command field defines the operation [1]:

- **ADD**: adds a new entry (replaces an existing one with the same match and priority);
- **MODIFY**: modifies the instructions of all entries matching the request;

- **MODIFY_STRICT**: modifies only the entry with exactly the same match and priority;
- **DELETE**: deletes all entries matching the request;
- **DELETE_STRICT**: deletes only the entry with exactly the same match and priority.

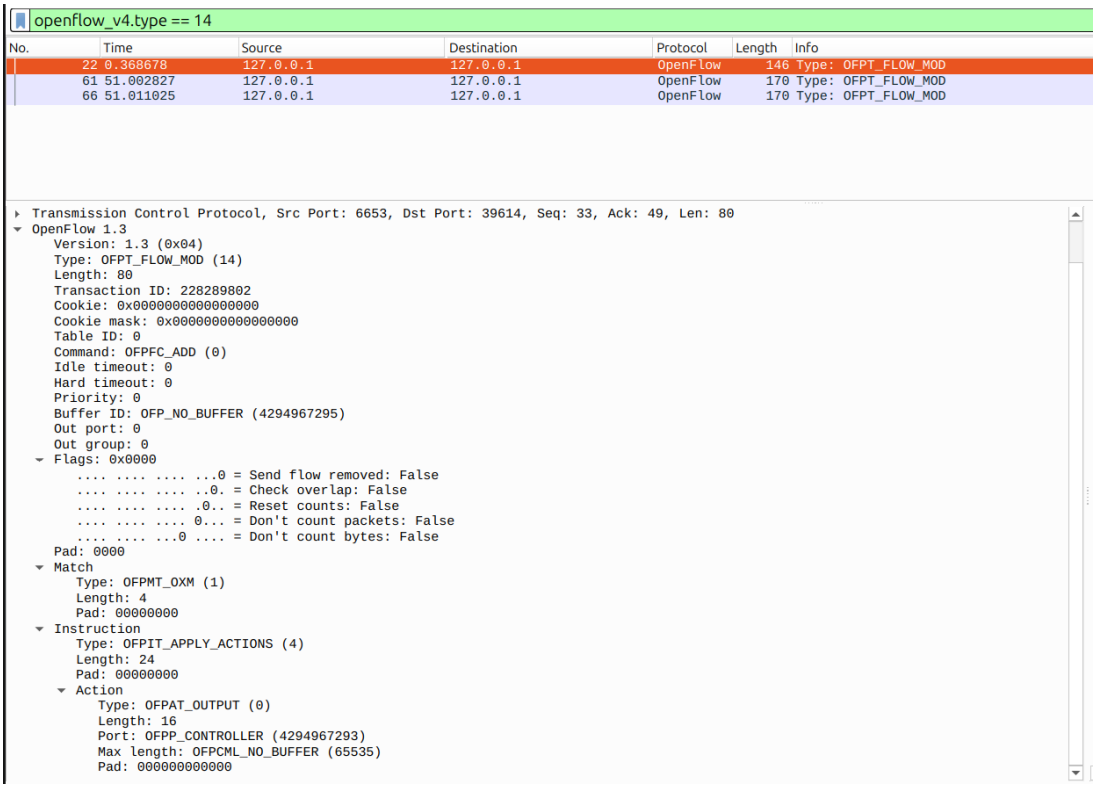


Figure 26: OFPT_FLOW_MOD installing the table-miss entry.

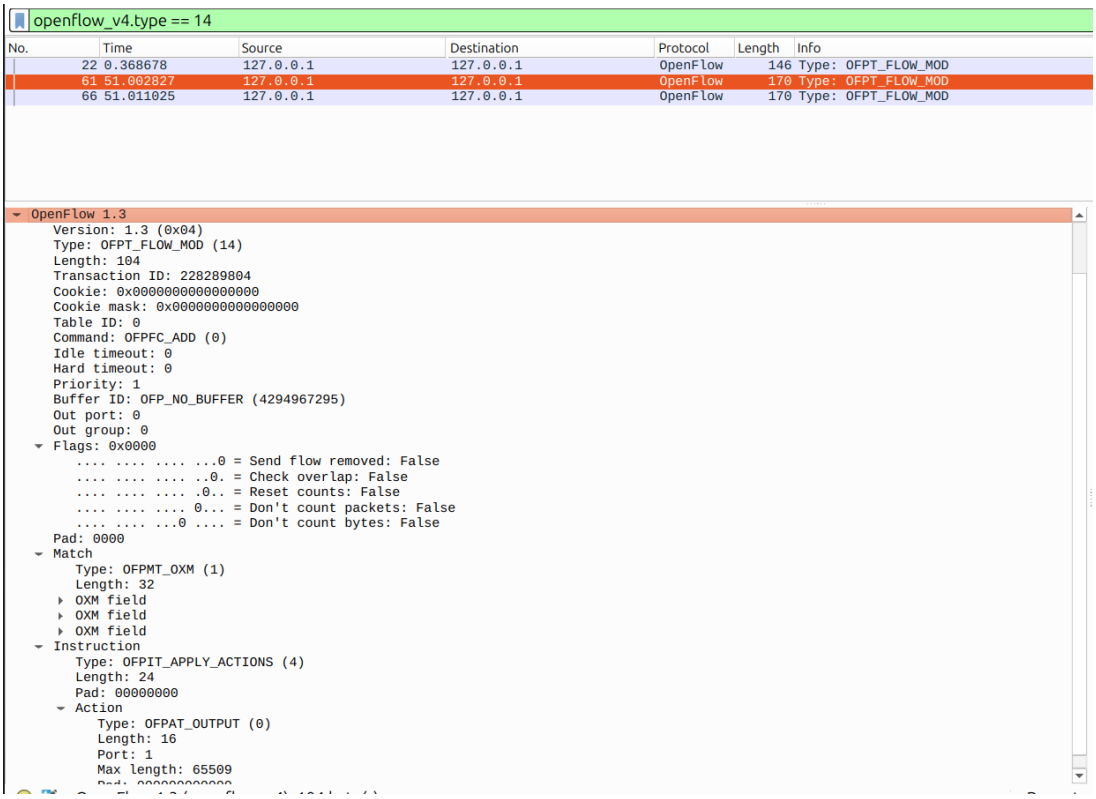


Figure 27: OFPT_FLOW_MOD installed reactively after a PACKET_IN.

Set async configuration

Asynchronous messages (PACKET_IN, FLOW_REMOVED, PORT_STATUS, ERROR) are sent by the switch without being requested [1]. No SET_ASYNC message was found in my capture, Ryu 4.34 does not send it by default, so the switch keeps its default configuration, in which the master controller receives the PACKET_IN messages needed by `simple_switch_13`.



Figure 28: No OFPT_SET_ASYNC message in the capture.

5 Exercise 4: dpctl and ovs-dpctl

5.1 Differences with the lab statement

dpctl vs. ovs-ofctl. `dpctl` is the management tool of the OpenFlow reference/software[5]. With Open vSwitch, the equivalent tool is `ovs-ofctl`, which provides the same commands and the same flow syntax [7]. Mininet's own `dpctl` CLI command is in fact a wrapper that runs `ovs-ofctl` on OVS switches [6]. `ovs-ofctl` was therefore used instead of `dpctl`.

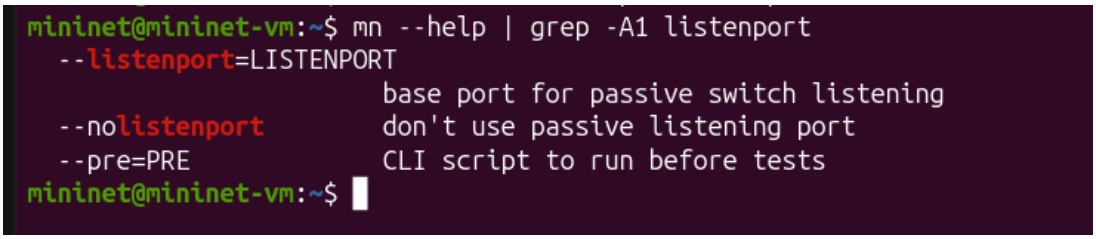


Figure 29: Default passive listening port of Mininet (`mn -help`).

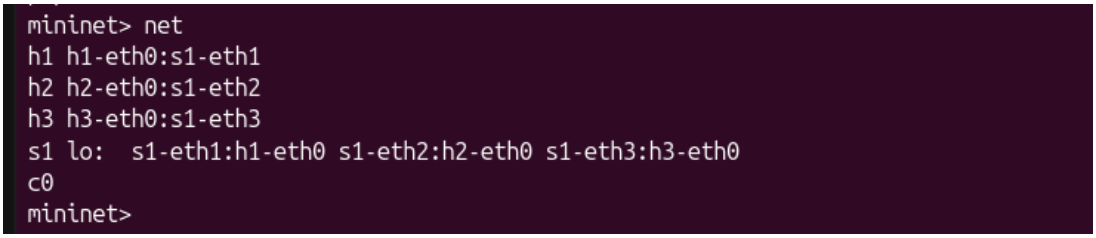


Figure 30: Mininet `net` command: host hi is connected to port s1-eth$i$.

5.2 Setup

The topology was created with the command of the lab statement, without starting any controller:

```
sudo mn --topo single,3 --mac --switch ovsk --controller remote
```

Mininet reports that the remote controller cannot be contacted (on ports 6653 and 6633), and `ovs-vsctl show` confirms that the switch is not connected to any controller, that it listens passively on port 6654 and that it is in secure fail mode.

`ovs-ofctl show` displays the datapath ID, the capabilities and the ports of the switch. `ovs-ofctl dump-flows` returns no entry: the flow table is empty.

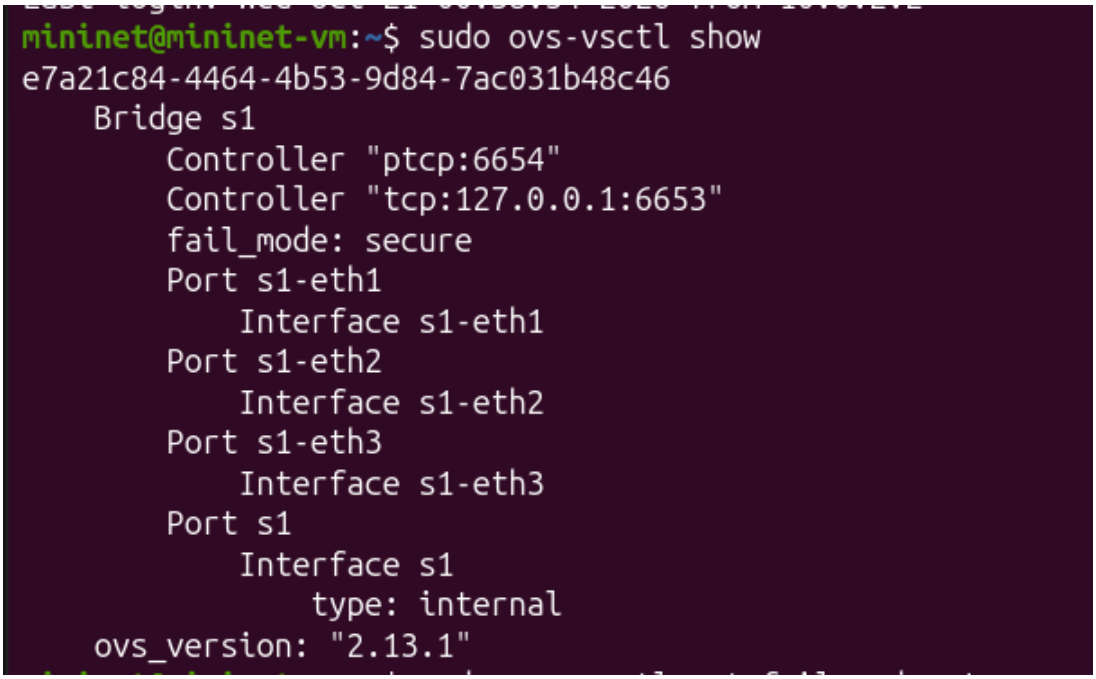


Figure 31: `ovs-vsctl show`: no controller connected, passive listening port 6654, secure fail mode.

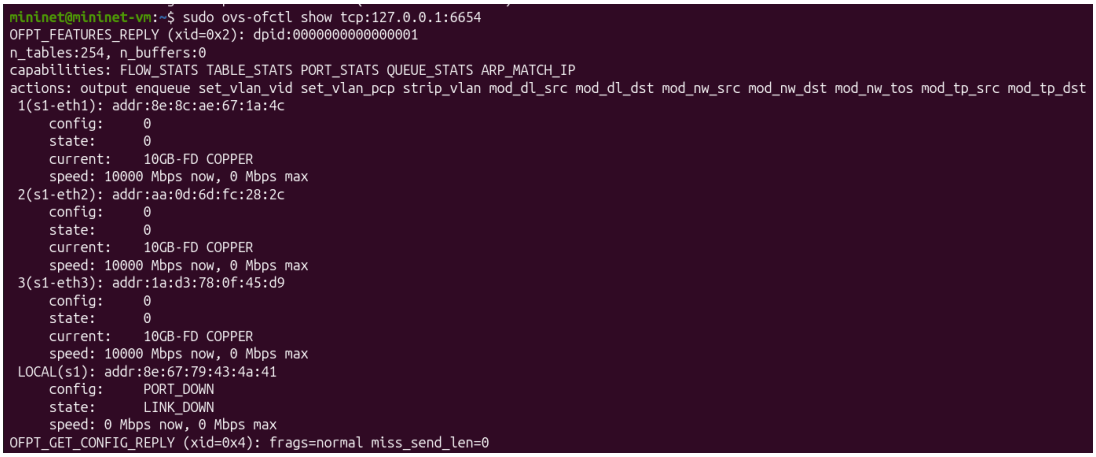


Figure 32: `ovs-ofctl show tcp:127.0.0.1:6654`: switch features and ports.

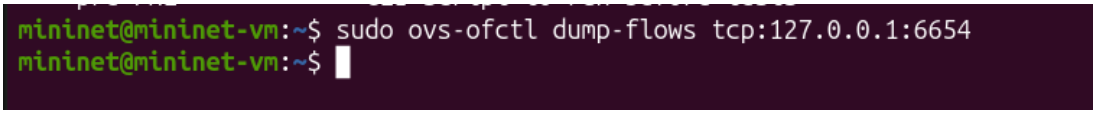


Figure 33: `ovs-ofctl dump-flows`: empty flow table.

5.3 Question 1

Do you get any replies? Why? No: `h1 ping -c3 h2` gives 100% packet loss. The flow table is empty and no controller is connected, so the switch has no rule for the incoming packets and cannot send them to a controller.

```
mininet@mininet-vm:~$ sudo mn --topo single,3 --mac --switch ovsk --controller remote
*** Creating network
*** Adding controller
Unable to contact the remote controller at 127.0.0.1:6653
Unable to contact the remote controller at 127.0.0.1:6633
Setting remote controller to 127.0.0.1:6653
*** Adding hosts:
h1 h2 h3
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1) (h3, s1)
*** Configuring hosts
h1 h2 h3
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet> h1 ping -c3 h3
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
From 10.0.0.1 icmp_seq=1 Destination Host Unreachable
From 10.0.0.1 icmp_seq=2 Destination Host Unreachable
From 10.0.0.1 icmp_seq=3 Destination Host Unreachable

--- 10.0.0.3 ping statistics ---
3 packets transmitted, 0 received, +3 errors, 100% packet loss, time 2032ms
pipe 3
mininet> h1 ping -c3 h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
From 10.0.0.1 icmp_seq=1 Destination Host Unreachable
From 10.0.0.1 icmp_seq=2 Destination Host Unreachable
From 10.0.0.1 icmp_seq=3 Destination Host Unreachable

--- 10.0.0.2 ping statistics ---
3 packets transmitted, 0 received, +3 errors, 100% packet loss, time 2027ms
```

Figure 34: `h1 ping -c3 h2` with an empty flow table: no reply.

5.4 Question 2

Two flow entries were added manually

```
mininet@mininet-vm:~$ sudo ovs-ofctl add-flow tcp:127.0.0.1:6654 in_port=1,actions=output:2
mininet@mininet-vm:~$ sudo ovs-ofctl add-flow tcp:127.0.0.1:6654 in_port=2,actions=output:1
```

Figure 35: The two flow entries.

```
mininet@mininet-vm:~$ sudo ovs-ofctl dump-flows tcp:127.0.0.1:6654
 cookie=0x0, duration=21.514s, table=0, n_packets=0, n_bytes=0, in_port="s1-eth1" actions=output:"s1-eth2"
 cookie=0x0, duration=6.988s, table=0, n_packets=0, n_bytes=0, in_port="s1-eth2" actions=output:"s1-eth1"
```

Figure 36: Flow table before adding the two entries manually.

Do you get any replies? Why? Yes: `h1 ping -c3 h2` now succeeds with 0% packet loss. The ARP request of h1 enters on port 1 and is forwarded to port 2 by the first entry, and the ARP reply of h2 comes back through the second entry, the ICMP echo requests and replies then follow the same two entries.

These entries only concern ports 1 and 2: `h1 ping h3` still fails, since no entry forwards traffic to or from port 3.



Figure 37: Flow table after adding the two entries manually.

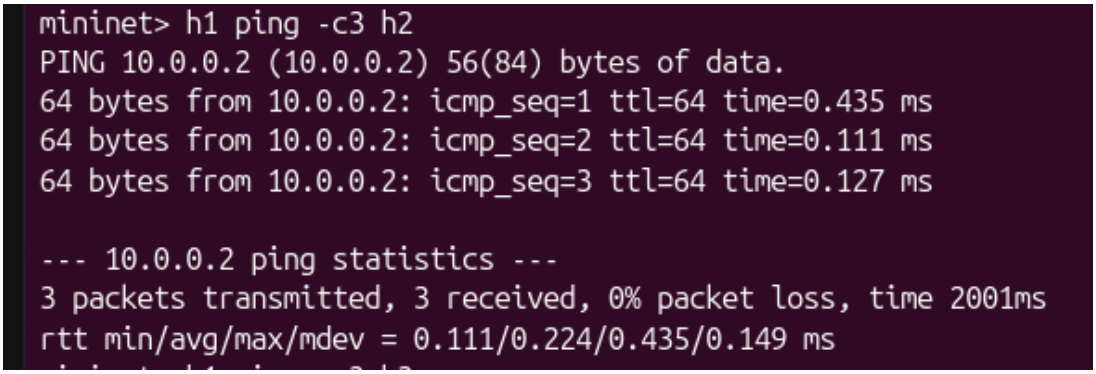


Figure 38: `h1 ping -c3 h2` after adding the flows: replies received.

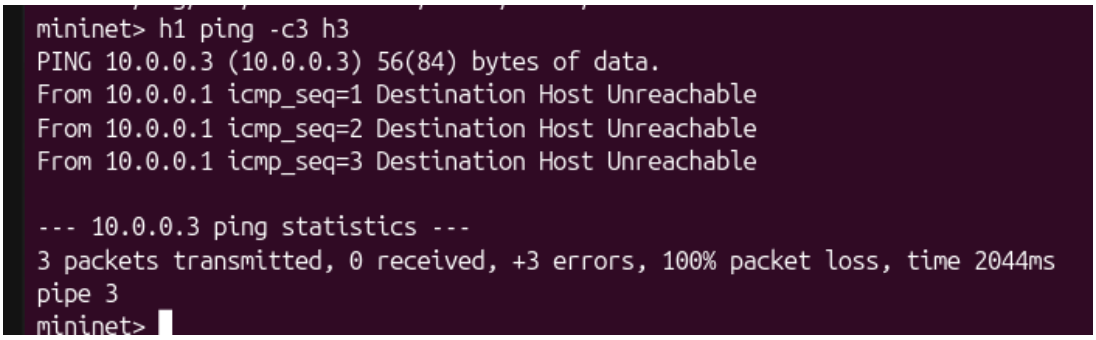


Figure 39: `h1 ping -c3 h3`: still no reply, no entry concerns port 3.

5.5 Question 3

Is this what you expected to see based on the ping traffic? Yes. After `h1 ping -c3 h2`, both entries show 5 packets and 378 bytes. This matches the traffic exactly: an ICMP echo frame is 98 bytes long and an ARP frame 42 bytes, and $3 \times 98 + 2 \times 42 = 378$. Each direction therefore carried 3 ICMP packets and 2 ARP packets:

- h1 to h2 (entry in_port=1): the ARP request of h1 (broadcast), the 3 ICMP echo requests and an ARP reply of h1;
- h2 to h1 (entry in_port=2): the ARP reply of h2, the 3 ICMP echo replies and an ARP request of h2.

The two counters are identical because every packet in one direction has a reply in the other.

The note of the statement about entries expiring after 60s does not apply here: the entries show no `idle_timeout`, since `ovs-ofctl add-flow` creates permanent entries by default [7].



Figure 40: Flow statistics after `h1 ping -c3 h2`: 5 packets and 378 bytes per entry.

6 Exercise 5: Topology discovery

6.1 Setup (OpenDaylight)

I install OpenDaylight Boron SR3 in the Mininet VM, with Java 8 as required by this release. The following features were installed:

```
feature:install odl-restconf odl-l2switch-switch-ui odl-mdsal-apidocs
feature:install odl-dlux-core odl-dlux-node
```

Once the features are installed, the controller listens on ports 6633 and 6653 (OpenFlow) and 8181 (web interface).

6.2 Question 1

The appropriate command:

```
sudo mn --topo linear,5 --mac --switch ovsk,protocols=OpenFlow13 --controller remote,
    ip=127.0.0.1,port=6653
```

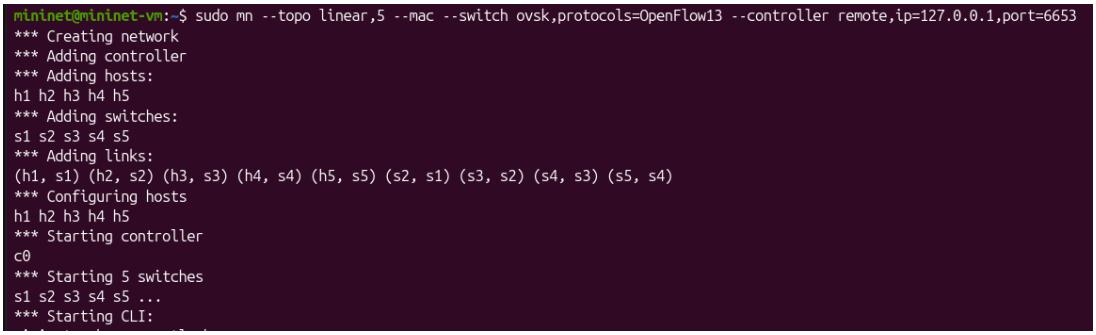


Figure 41: Mininet start-up: linear topology with five switches, five hosts and nine links.

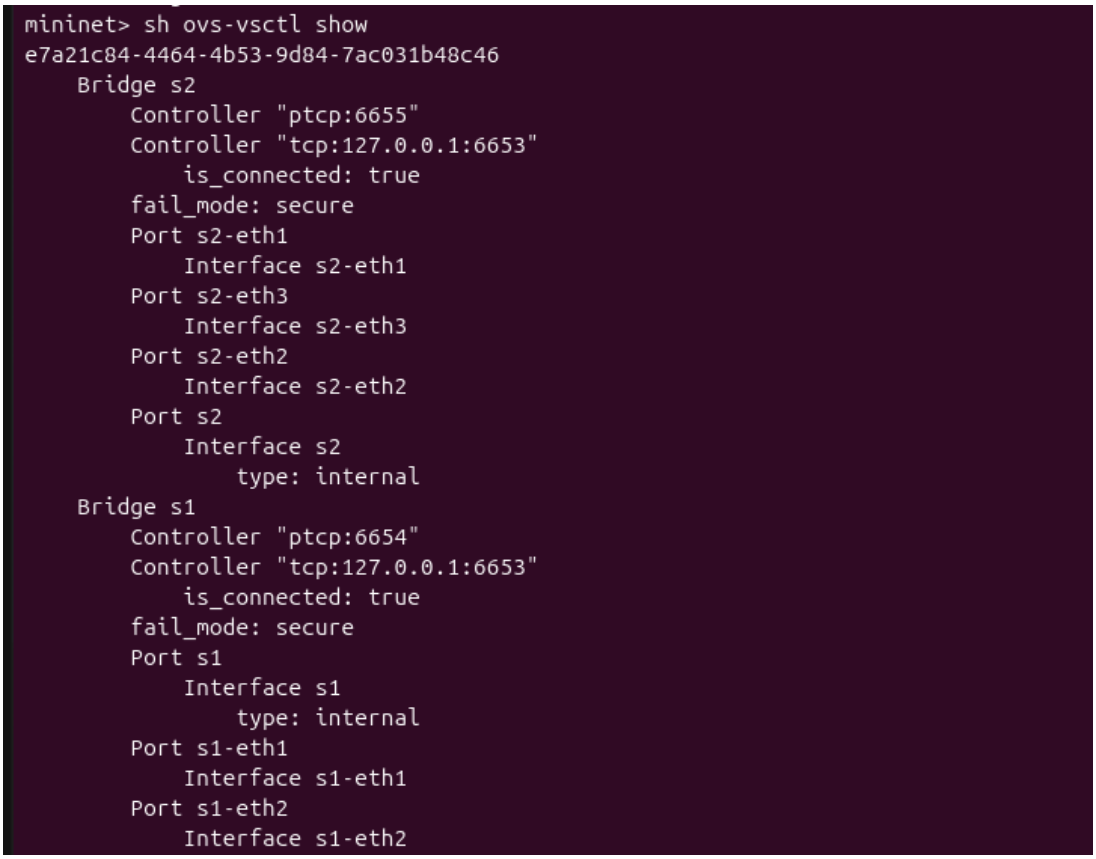


Figure 42: `ovs-vsctl show` (extract): the switches are connected to OpenDaylight.

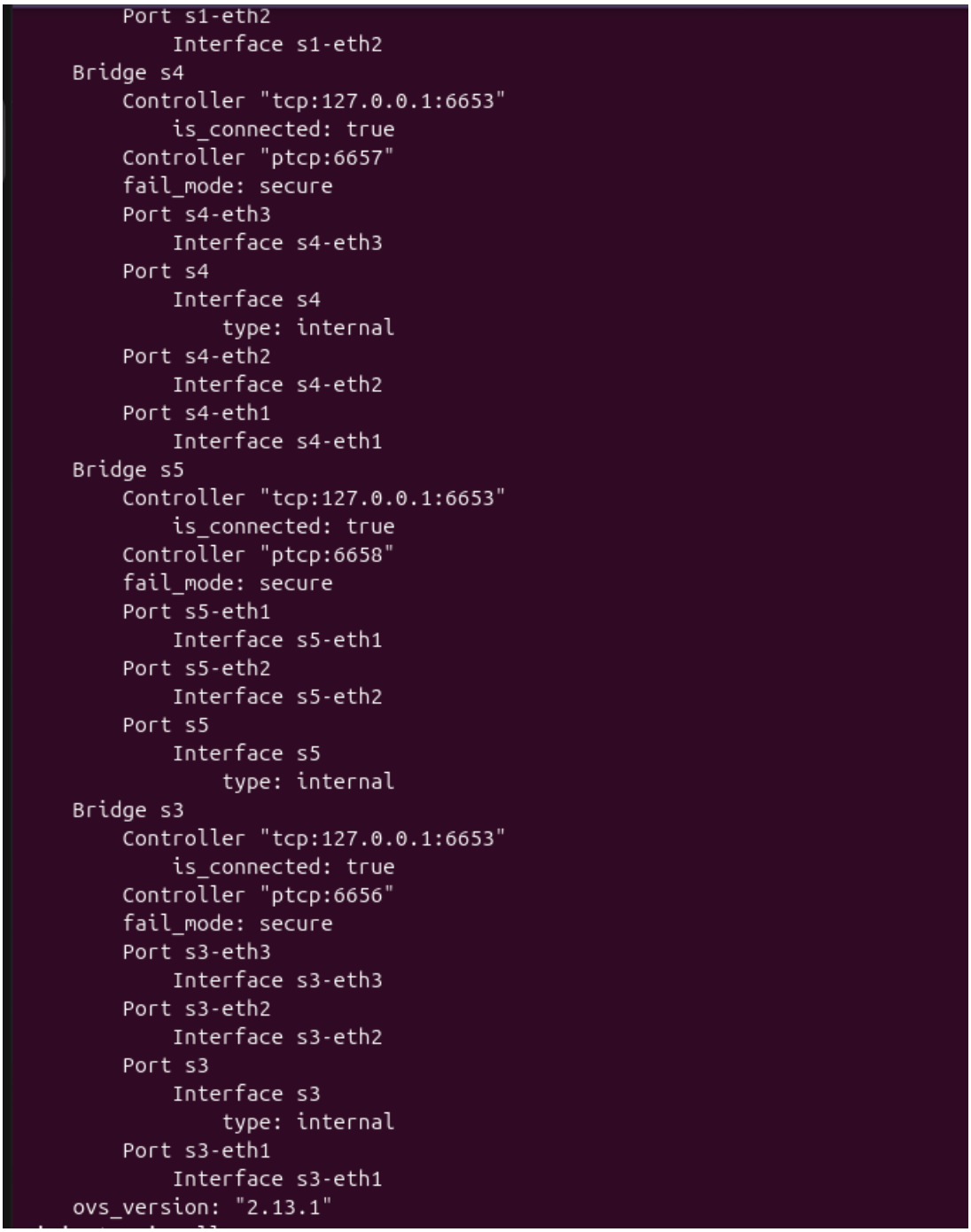


Figure 43: `ovs-vsctl show` (extract): the switches are connected to OpenDaylight.

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4 h5
h2 -> h1 h3 h4 h5
h3 -> h1 h2 h4 h5
h4 -> h1 h2 h3 h5
h5 -> h1 h2 h3 h4
*** Results: 0% dropped (20/20 received)
```

Figure 44: `pingall` with OpenDaylight: 0% dropped.

6.3 Question 2

In wireshark the IPv4 traffic is hidden and the LLDP frames appear.

The Ethernet header of these frames shows:

- an **ethertype of 0x88cc**, the standard value assigned to LLDP by IEEE 802.1AB [11], which identifies the payload as an LLDP data unit,
- a **destination MAC address of 01:23:00:00:00:01**. Its I/G bit is set, so it is a group multicast address. However, it is not the standard LLDP address defined by IEEE 802.1AB [11]: it is a non-standard address used by OpenDaylight.

With the standard address, the one-hop scope of LLDP is guaranteed by the bridges themselves. In an OpenFlow network, it is guaranteed by the controller: a flow entry sends every frame with ethertype 0x88cc to the controller, so a switch that receives an LLDP frame from its neighbour does not forward it but reports it to the controller.

The source addresses alternate between two locally administered MAC addresses: LLDP frames are sent in both directions on the link, by s1 towards s2 and by s2 towards s1.

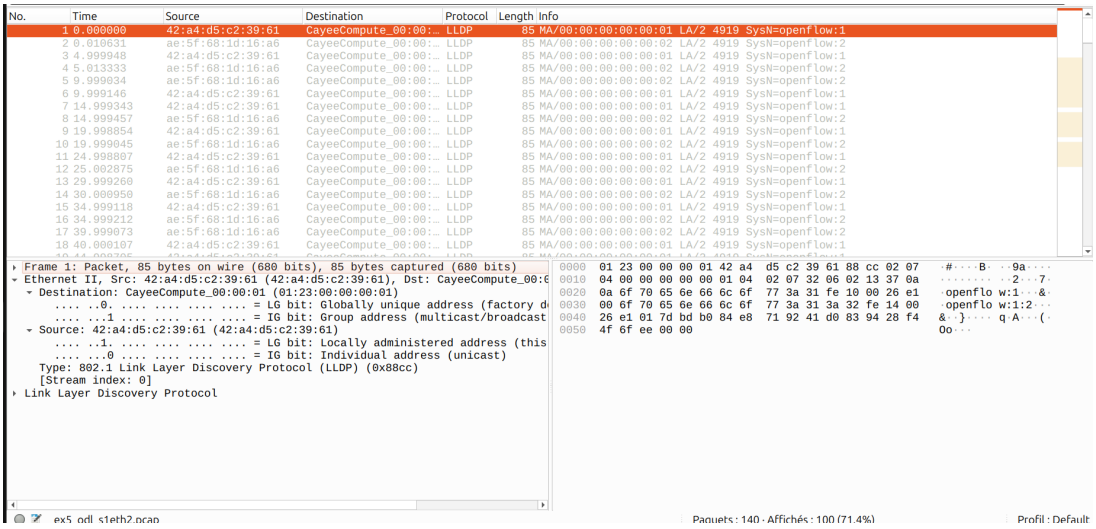


Figure 45: LLDP frames captured on `s1-eth2` with OpenDaylight (filter `!(ip.version==4)`).

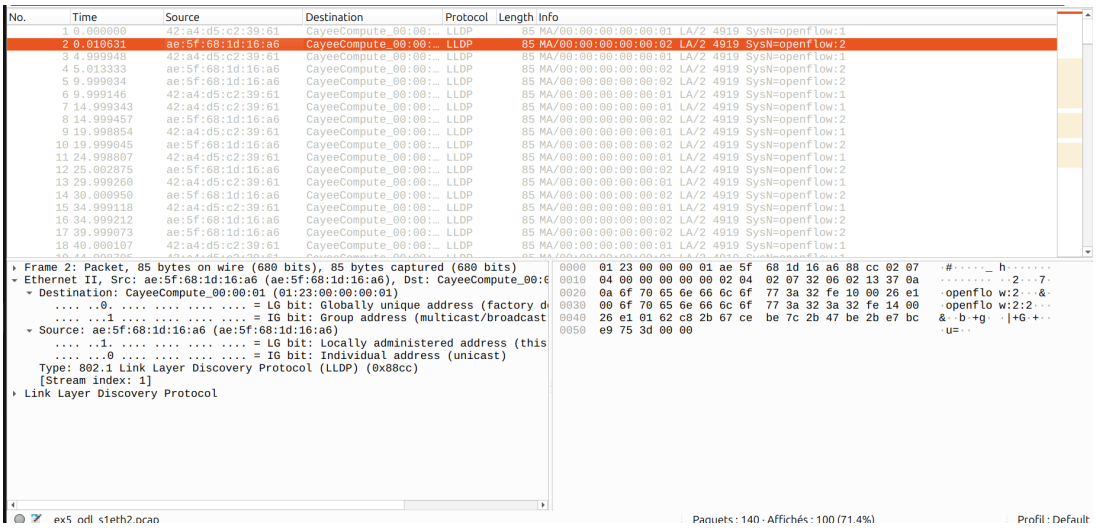


Figure 46: Ethernet header of an LLDP frame sent with OpenDaylight: ethertype 0x88cc, destination 01:23:00:00:00:01.

6.4 Question 3

An LLDP data unit is a sequence of TLVs (Type–Length–Value). Each TLV starts with a 2-byte header made of a 7-bit *type* and a 9-bit *length*, followed by a value of *length* bytes, which often starts with a subtype giving its format [11].

The LLDP frames sent by OpenDaylight contain **7 TLVs**.

Type	**TLV**	**Length**	**Value**
1	Chassis ID	7	MAC address 00:00:00:00:00:01 (switch s1)
2	Port ID	2	locally assigned, “2” (port s1-eth2)
3	Time To Live	2	4919 s (0x1337)
5	System Name	10	`openflow:1`
127	Organisation-specific	16	OUI 00:26:e1, subtype 0: `openflow:1:2`
127	Organisation-specific	20	OUI 00:26:e1, subtype 1: 16 bytes
0	End of LLDPDU	0	–

Table 1: TLVs of an LLDP frame sent by OpenDaylight from s1, port 2.

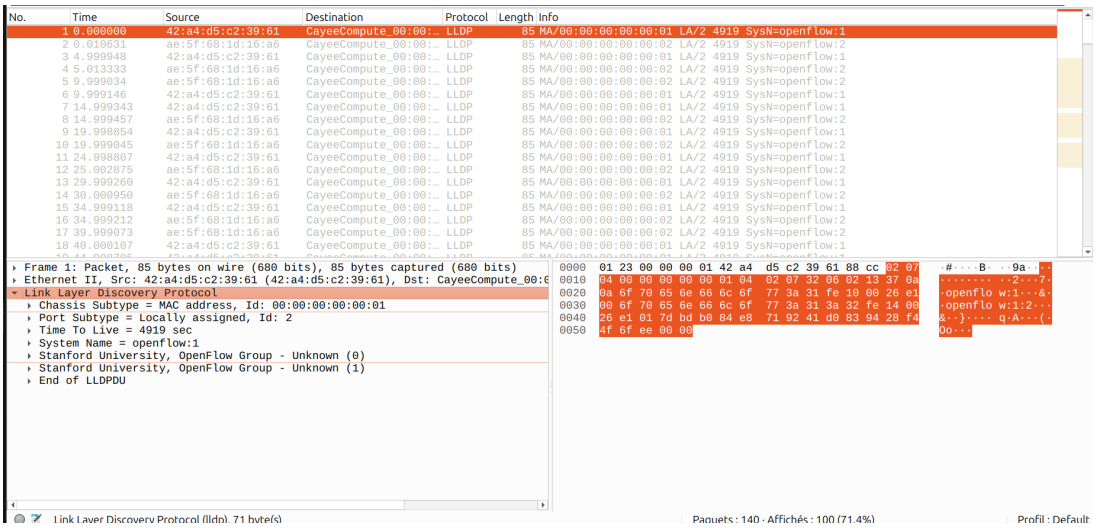


Figure 47: TLVs of an LLDP frame sent by OpenDaylight.

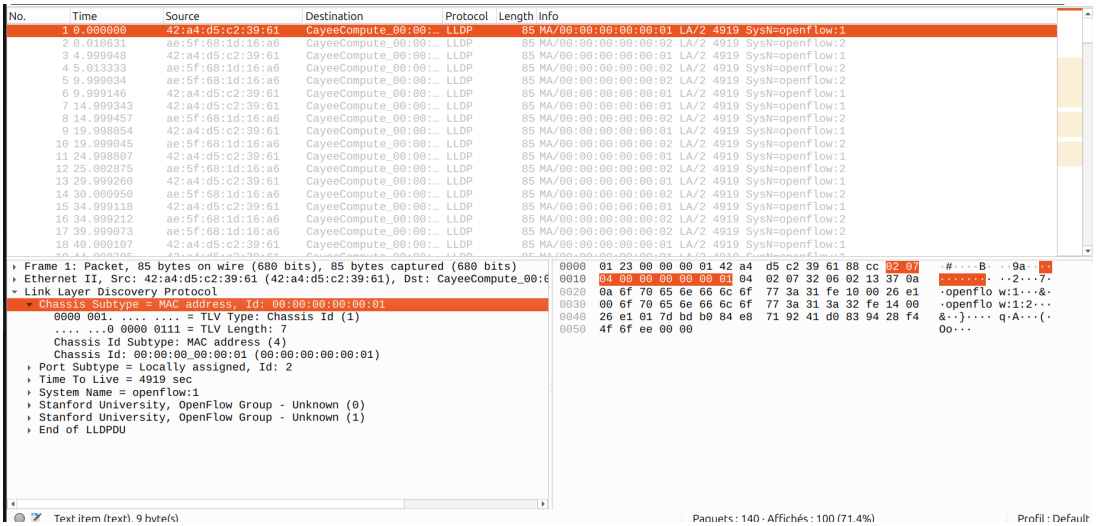


Figure 48: Detail of the Chassis ID TLV: 7-bit type and 9-bit length.

6.5 Question 4

With the filter lldp on the loopback interface, we see OpenFlow messages. The switches do not create LLDP frames: the controller creates them and sends them inside OpenFlow messages.

A **PACKET_OUT** is sent by the controller to a switch. It asks the switch to send a packet on a port. In packet 286 it contains an OUTPUT action to port 3 and the LLDP frame of switch 3 (`openflow:3:3`).

A **PACKET_IN** is sent by a switch to the controller. It reports a packet received by the switch. In packet 339 (Figure 51), it contains the reason (`OFPR_ACTION`, so a flow rule sends LLDP to the controller), the input port (port 2) and the LLDP frame. This frame was sent by switch 2 on port 2 and received by switch 1 on port 2. So the controller learns that there is a link between these two ports.

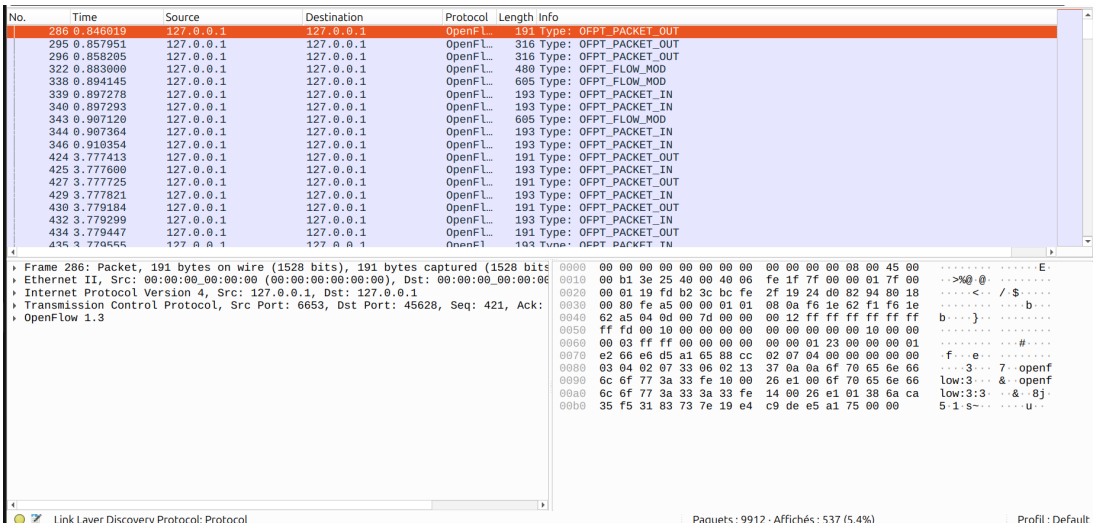


Figure 49: OpenFlow messages with LLDP frames (filter `lldp`).

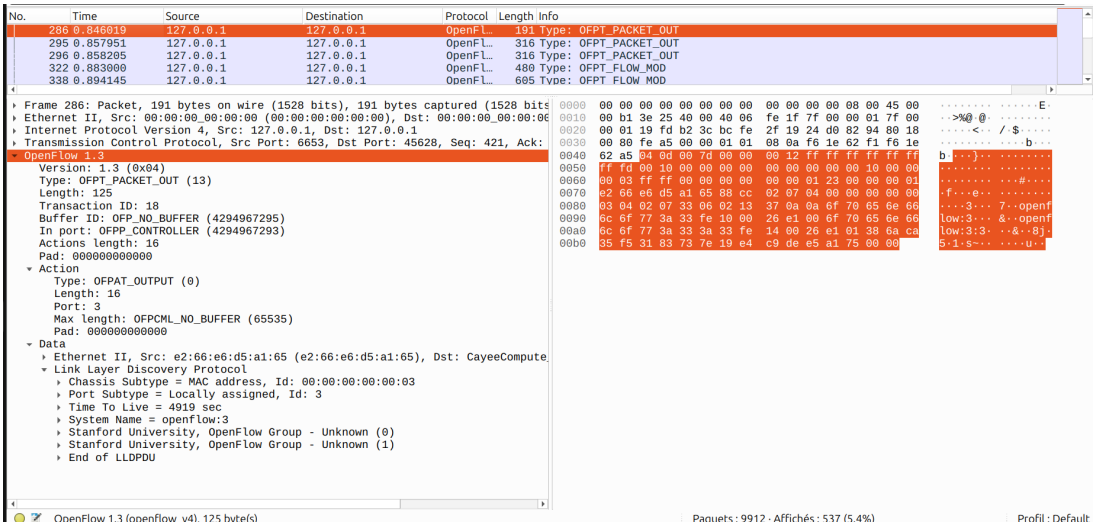


Figure 50: PACKET_OUT: output to port 3 with an LLDP frame.

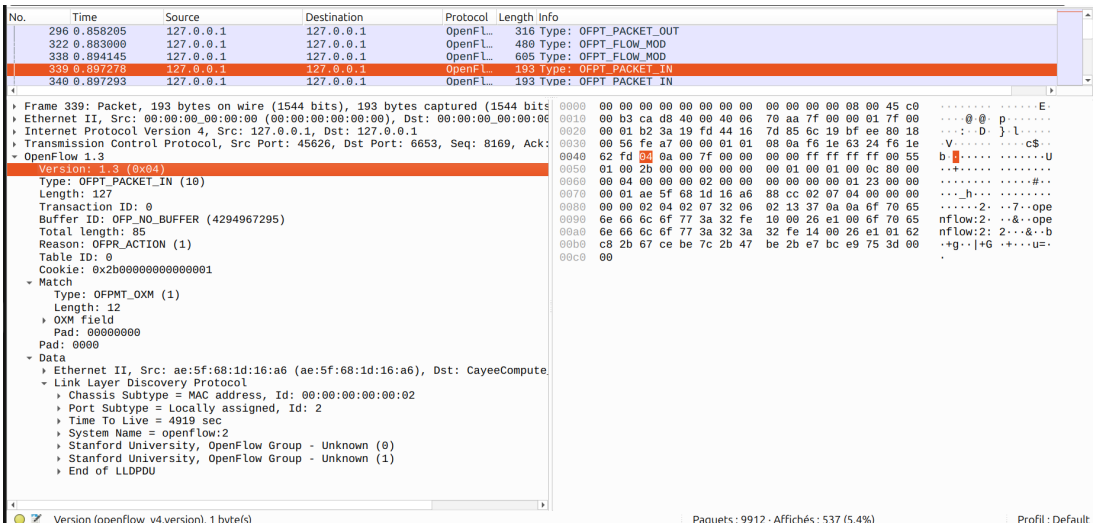


Figure 51: PACKET_IN: LLDP frame received on port 2.

6.6 Question 5

In the capture on `s1-eth2`, each switch sends one LLDP frame every 5 seconds (Figure 52). So the default period used by OpenDaylight is 5 s.

If we send LLDP more often, the controller detects link failures faster, but there are more messages to process. If we send LLDP less often, there are fewer messages, but the controller sees failures later. In a big network with many ports, this choice is important.

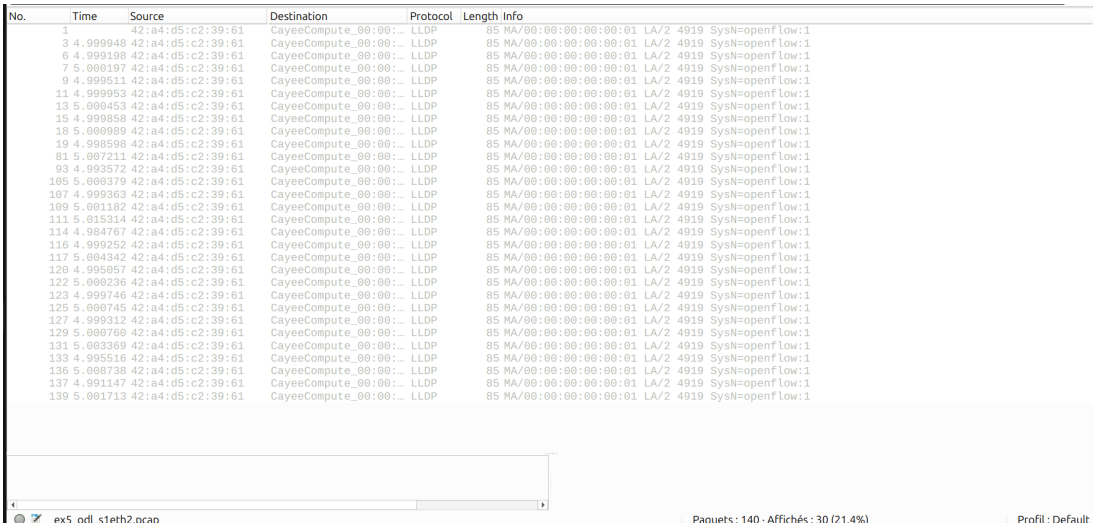


Figure 52: Time between two LLDP frames sent by s1: about 5 s.

6.7 Question 6

The controller discovers the topology in a few steps:

1. When a switch connects, the controller learns its ID and its ports (FEATURES and PORT_DESC messages). It does not know the links yet.

2. The controller installs a rule in each switch to send LLDP frames to the controller.

3. The controller sends an LLDP frame on each port of each switch with a PACKET_OUT. The LLDP frame says which switch and which port it comes from.

4. The neighbour switch receives this frame and sends it back to the controller in a PACKET_IN, with the port where it was received.

5. The controller now knows the two ends of the link, so it adds this link to the topology.

This is repeated every 5 seconds. If the controller does not receive LLDP on a link anymore, it considers that the link is down.

6.8 Setup (ONOS)

ONOS 2.7.0 was run in a Docker container, as suggested in [12]. The option `-network host` was used, so ONOS uses the network of the VM: Mininet connects to 127.0.0.1:6653.

```
sudo docker run -d --name onos --network host onosproject/onos:2.7.0
```

`openflow` adds the OpenFlow support and the LLDP link provider topology discovery. `fwd` adds reactive forwarding, so hosts can ping each other. ONOS shows 5 devices, 8 links and 5 hosts. The 8 links are the 4 links between switches, counted in both directions.

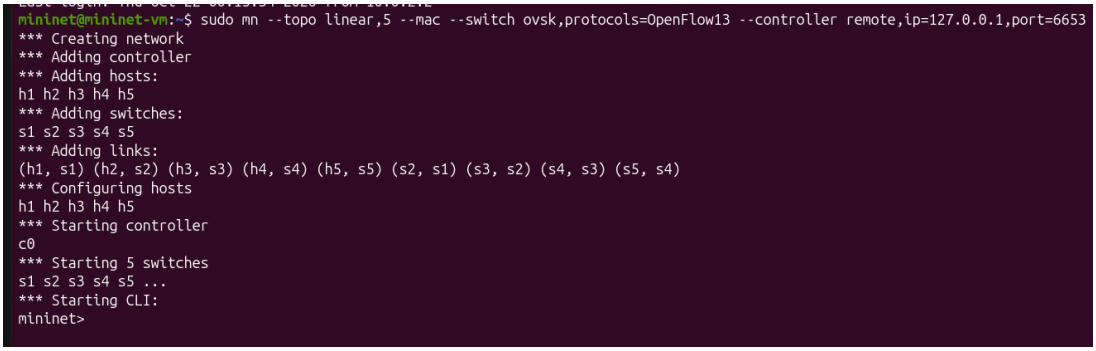


Figure 53: `ovs-vsctl show` (extract): the switches are connected to ONOS.

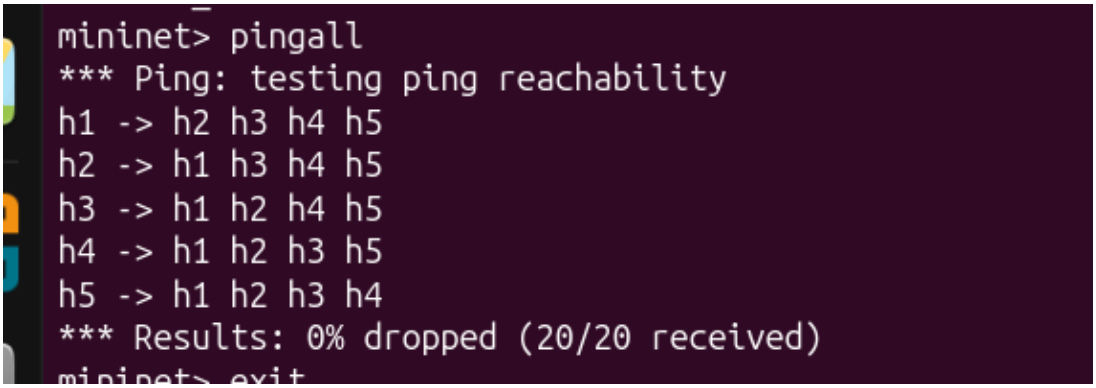


Figure 54: `pingall` with ONOS: 0% dropped.

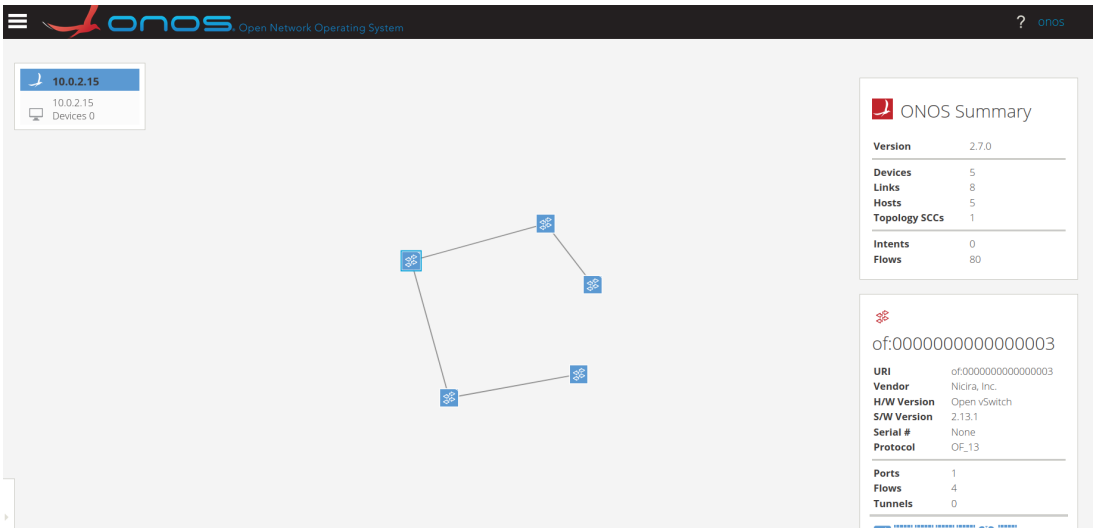


Figure 55: Topology discovered by ONOS (web interface).

6.9 Question 2 (ONOS)

With ONOS, two kinds of frames are sent in pairs on `s1-eth2` (Figure 56):

- **LLDP**: ethertype 0x88cc and destination 01:80:c2:00:00:0e (Figure 57). This is the standard LLDP address. Switches do not forward it, so the frame only goes to the direct neighbour.
- **BDDP**: ethertype 0x8942 and destination ff:ff:ff:ff:ff:ff (Figure 58). 0x8942 is not a standard ethertype, so Wireshark shows it as "Unknown", but the content is the same as the LLDP frame. The destination is broadcast, so normal switches forward it. This is why BDDP can find links between OpenFlow switches through non-OpenFlow switches (hybrid network).

Unlike OpenDaylight, ONOS uses the standard LLDP address and also sends BDDP. All the frames have the same source MAC address (02:eb:9f:67:c9:42), even when they are sent by different switches.

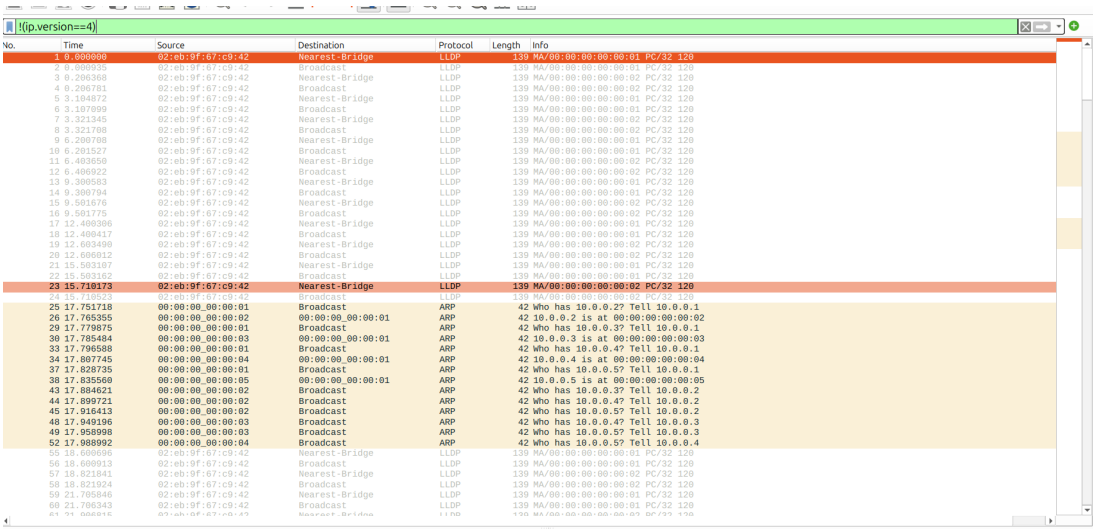
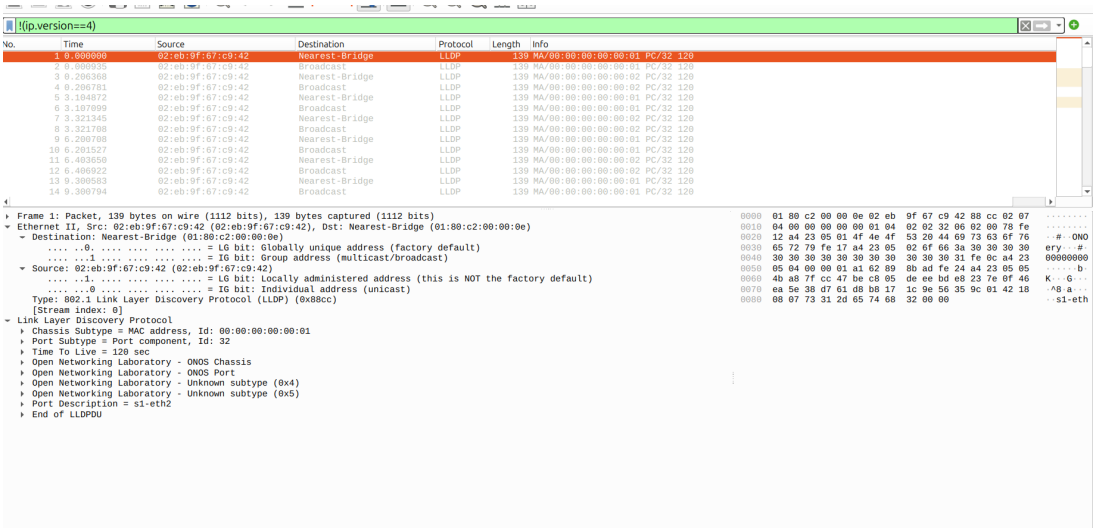


Figure 56: LLDP and BDDP frames sent by ONOS on `s1-eth2`.


!(ip.version==4)
No. Time Source Destination Protocol Length Info
1 0.000000 02:eb:9f:67:c9:42 Nearest-Bridge LLDP 139 MA/00:00:00:00:00:01 PC/32 120
2 0.000935 02:eb:9f:67:c9:42 Broadcast LLDP 139 MA/00:00:00:00:00:01 PC/32 120
3 0.206368 02:eb:9f:67:c9:42 Nearest-Bridge LLDP 139 MA/00:00:00:00:00:02 PC/32 120
4 0.206781 02:eb:9f:67:c9:42 Broadcast LLDP 139 MA/00:00:00:00:00:02 PC/32 120
5 3.104872 02:eb:9f:67:c9:42 Nearest-Bridge LLDP 139 MA/00:00:00:00:00:01 PC/32 120
6 3.107099 02:eb:9f:67:c9:42 Broadcast LLDP 139 MA/00:00:00:00:00:01 PC/32 120
7 3.321345 02:eb:9f:67:c9:42 Nearest-Bridge LLDP 139 MA/00:00:00:00:00:02 PC/32 120
8 3.321708 02:eb:9f:67:c9:42 Broadcast LLDP 139 MA/00:00:00:00:00:02 PC/32 120
9 6.200708 02:eb:9f:67:c9:42 Nearest-Bridge LLDP 139 MA/00:00:00:00:00:01 PC/32 120
10 6.201527 02:eb:9f:67:c9:42 Broadcast LLDP 139 MA/00:00:00:00:00:01 PC/32 120
11 6.403650 02:eb:9f:67:c9:42 Nearest-Bridge LLDP 139 MA/00:00:00:00:00:02 PC/32 120
12 6.406922 02:eb:9f:67:c9:42 Broadcast LLDP 139 MA/00:00:00:00:00:02 PC/32 120
13 9.300583 02:eb:9f:67:c9:42 Nearest-Bridge LLDP 139 MA/00:00:00:00:00:01 PC/32 120
14 9.300794 02:eb:9f:67:c9:42 Broadcast LLDP 139 MA/00:00:00:00:00:01 PC/32 120
Frame 1: Packet, 139 bytes on wire (1112 bits), 139 bytes captured (1112 bits)
Ethernet II, Src: 02:eb:9f:67:c9:42 (02:eb:9f:67:c9:42), Dst: Nearest-Bridge (01:80:c2:00:00:0e)
Destination: Nearest-Bridge (01:80:c2:00:00:0e)
.... ..0. .... = LG bit: Globally unique address (factory default)
....1 .... = IG bit: Group address (multicast/broadcast)
Source: 02:eb:9f:67:c9:42 (02:eb:9f:67:c9:42)
.... ..1. .... = LG bit: Locally administered address (this is NOT the factory default)
....0 .... = IG bit: Individual address (unicast)
Type: 802.1 Link Layer Discovery Protocol (LLDP) (0x88cc)
[Stream index: 0]
Link Layer Discovery Protocol
Chassis Subtype = MAC address, Id: 00:00:00:00:00:01
Port Subtype = Port component, Id: 32
Time To Live = 120 sec
Open Networking Laboratory - ONOS Chassis
Open Networking Laboratory - ONOS Port
Open Networking Laboratory - Unknown subtype (0x4)
Open Networking Laboratory - Unknown subtype (0x5)
Port Description = s1-eth2
End of LLDPDU
0000 01 80 c2 00 00 0e 02 eb 9f 67 c9 42 88 cc 02 07
0010 04 00 00 00 00 00 01 04 02 02 32 06 02 00 78 fe
0020 12 a4 23 05 01 4f 4e 4f 53 20 44 69 73 63 6f 76
0030 65 72 79 fe 17 a4 23 05 02 6f 66 3a 30 30 30 30
0040 30 30 30 30 30 30 30 30 30 30 30 31 fe 0c a4 23
0050 05 04 00 00 01 a1 62 89 8b ad fe 24 a4 23 05 05
0060 4b a8 7f cc 47 be c8 05 de ee bd e8 23 7e 0f 46
0070 ea 5e 38 d7 61 d8 b8 17 1c 9e 56 35 9c 01 42 18
0080 08 07 73 31 2d 65 74 68 32 00 00
....

Type	TLV	Length	Value
1	Chassis ID	7	MAC 00:00:00:00:00:01 (switch s1)
2	Port ID	2	port component, “2” (port 2)
3	Time To Live	2	120 s
127	Org. specific	18	OUI a4:23:05, subtype 1 (“ONOS Chassis”): “ONOS Discovery”
127	Org. specific	23	subtype 2 (“ONOS Port”): `of:0000000000000001`
127	Org. specific	12	subtype 4 (unknown): 000001a162898bad
127	Org. specific	36	subtype 5 (unknown): 32 bytes
4	Port Description	7	`s1-eth2`
0	End of LLDPDU	0	–

Table 2: TLVs of an LLDP frame sent by ONOS from s1, port 2.

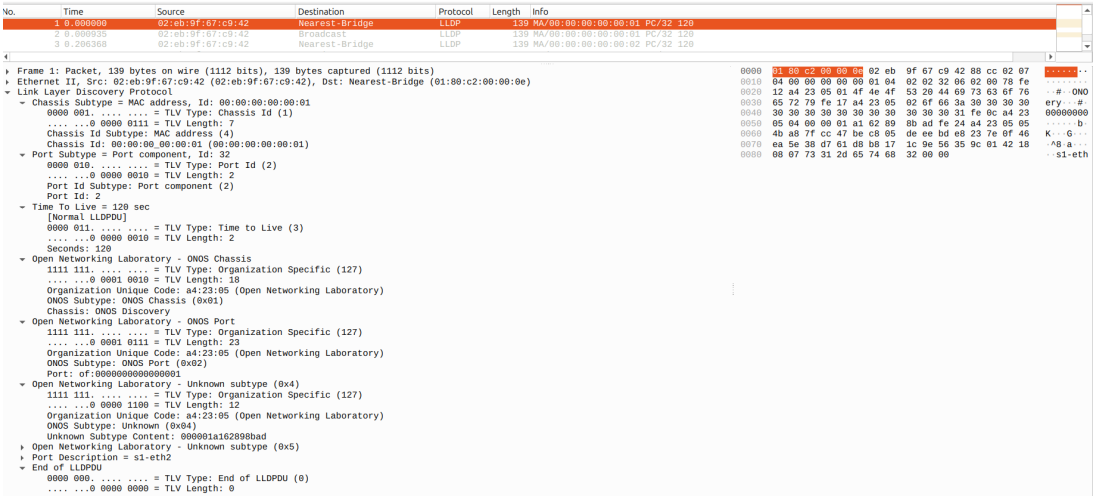


Figure 59: TLVs of an LLDP frame sent by ONOS.

6.11 Question 4 (ONOS)

ONOS uses PACKET_OUT and PACKET_IN in the same way as OpenDaylight (Figure 60).

A **PACKET_OUT** asks a switch to send the frame on one port. With ONOS, there are two PACKET_OUT messages for each port: one with an LLDP frame and one with a BDDP frame.

A **PACKET_IN** reports a frame received by a switch. In frame 197, the reason is `OFPR_ACTION`, so a flow rule sends LLDP to the controller, like with ODL. The input port is 2 and the data is an LLDP frame sent by switch 2 on port 2. This PACKET_IN comes from s1, so ONOS learns the link between s2 port 2 and s1 port 2.

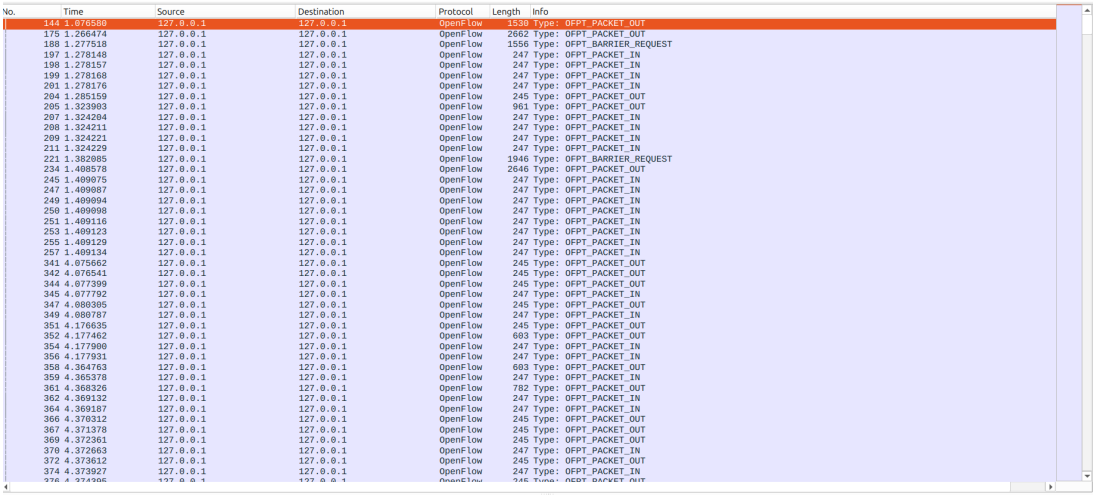


Figure 60: OpenFlow messages with LLDP frames sent by ONOS.

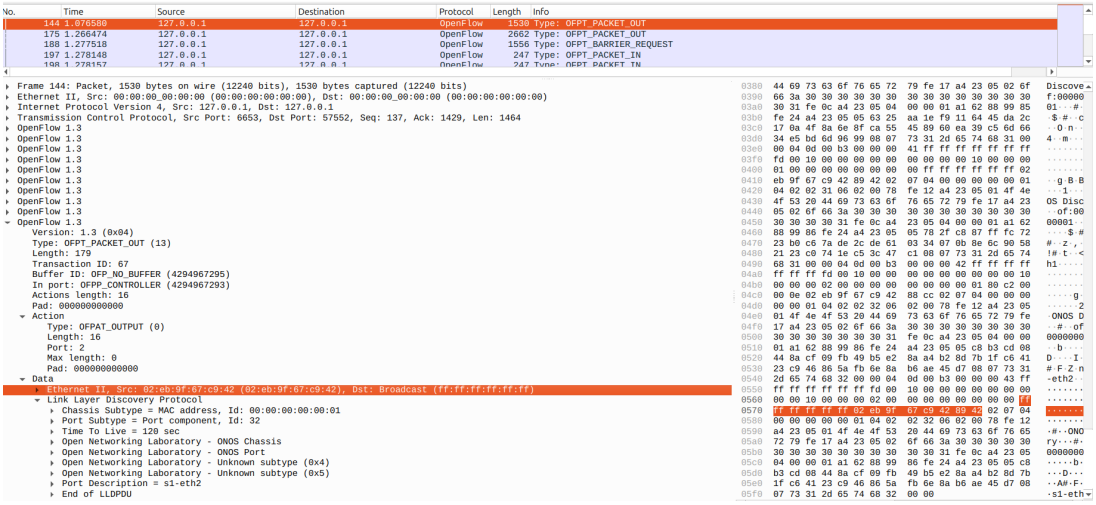


Figure 61: PACKET_OUT sent by ONOS with an LLDP frame.

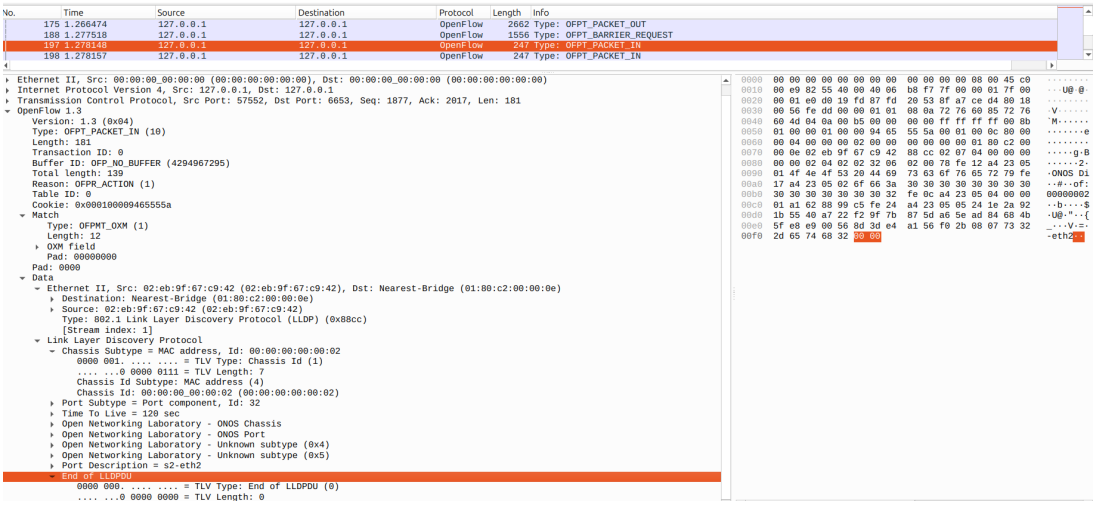


Figure 62: PACKET_IN received by ONOS: LLDP frame of switch 2 received on port 2.

6.12 Question 5 (ONOS)

With ONOS, each switch sends an LLDP frame (and a BDDP frame) on the link about every 3 seconds (Figure 63). So ONOS probes the links more often than OpenDaylight (5 s). The topology is updated faster, but there are more messages: two PACKET_OUT per port (LLDP and BDDP) every 3 seconds.

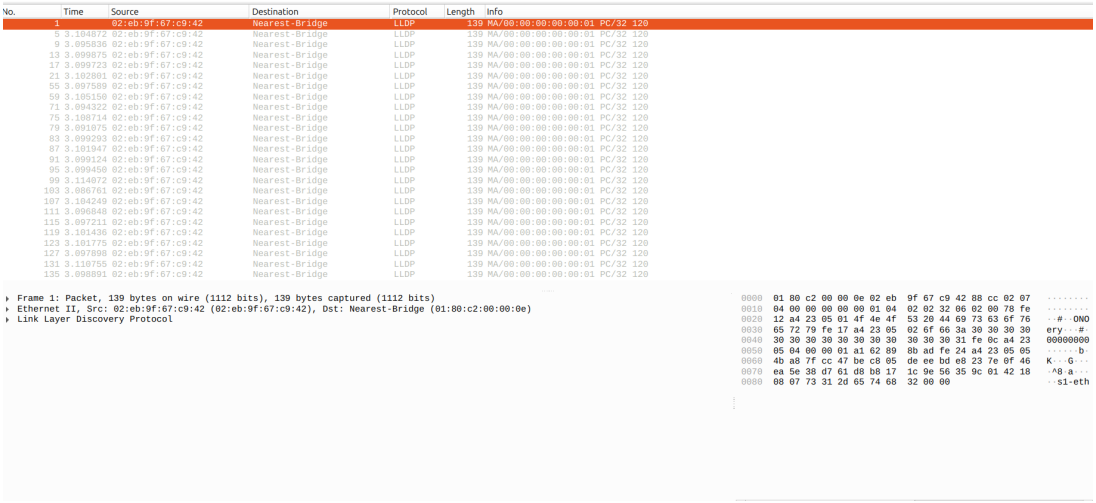


Figure 63: Time between two LLDP frames sent by s1 with ONOS: about 3 s.

6.13 Question 6 (ONOS)

ONOS discovers the topology in the same way as ODL: it installs a rule to send LLDP to the controller, sends an LLDP frame on each port with a PACKET_OUT, and the neighbour switch sends it back in a PACKET_IN with the input port. The controller then knows the two ends of the link.

There are some differences with OpenDaylight: ONOS also sends BDDP frames, to find links through non-OpenFlow switches, it uses the standard LLDP address, and it sends the probes every 3 seconds instead of 5. ONOS counts each link in both directions: the 4 links between the switches give 8 links in the `links` command (Figure 64) and in the web interface.

```
karaf@root > links
src=of:0000000000000001/2, dst=of:0000000000000002/2, type=DIRECT, state=ACTIVE, expected=false
src=of:0000000000000002/2, dst=of:0000000000000001/2, type=DIRECT, state=ACTIVE, expected=false
src=of:0000000000000002/3, dst=of:0000000000000003/2, type=DIRECT, state=ACTIVE, expected=false
src=of:0000000000000003/2, dst=of:0000000000000002/3, type=DIRECT, state=ACTIVE, expected=false
src=of:0000000000000003/3, dst=of:0000000000000004/2, type=DIRECT, state=ACTIVE, expected=false
src=of:0000000000000004/2, dst=of:0000000000000003/3, type=DIRECT, state=ACTIVE, expected=false
src=of:0000000000000004/3, dst=of:0000000000000005/2, type=DIRECT, state=ACTIVE, expected=false
src=of:0000000000000005/2, dst=of:0000000000000004/3, type=DIRECT, state=ACTIVE, expected=false
karaf@root >
```

Figure 64: Links discovered by ONOS (`links` command).

7 Exercise 6: Switching rules

I use ONOS as the controller.

7.1 Question 1

The topology with two switches, using OpenFlow v4 (OpenFlow 1.3), is created `linear,2` creates two switches connected together, with one host on each switch. The OpenFlow traffic is captured on the loopback interface.

```
mininet@mininet-vm:~$ sudo mn --topo linear,2 --mac --switch ovsk,protocols=OpenFlow13 --controller remote,ip=127.0.0.1,port=6653
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
s1 s2
*** Adding links:
(h1, s1) (h2, s2) (s2, s1)
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 2 switches
s1 s2 ...
*** Starting CLI:
mininet>
```

Figure 65: Mininet start-up: two switches connected to ONOS.

7.2 Question 3

The filter `openflow_v4.type==14` shows the FLOW_MOD messages. They are sent by ONOS just after the connection of each switch, to install its default rules.

7.3 Question 4

Just after the start, s2 contains 4 default rules (Figure 66). All of them send the packets to the CONTROLLER:

- BDDP (`dl_type=0x8942`) and LLDP (`dl_type=0x88cc`), priority 40000: topology discovery;
- ARP, priority 40000: host discovery and ARP handling;
- IPv4 (`ip`), priority 5: reactive forwarding of the first IP packets.

The discovery rules have a very high priority, so they are always applied first. These rules have no timeout and the flag `send_flow_rem`. At the beginning, the switch knows nothing, so it must send these packets to the controller.

7.4 Question 6

`pingall` gives 0% dropped. The first ARP and IP packets match the default rules and are sent to ONOS (PACKET_IN). ONOS then installs new rules with FLOW_MOD, and the next packets are forwarded directly by the switches.

7.5 Question 7

Yes. After the ping, 14 new rules appear on s2. They have priority 10 and match the input port, the source MAC and the destination MAC, with an output action to the right port. They are added by the forwarding application because s2 is on the path between h1/h2 and h3/h4/h5. Their priority (10) is higher than the IPv4 rule (5), so the next packets do not go to the controller.

7.6 Question 8

10.0.0.10 does not exist, so h1 never gets an ARP reply and never sends an ICMP packet: it only sends ARP requests. On s1, the counter of the **ARP rule** goes from 22 to 31 packets, while the IPv4 rule stays at 8 packets. So the ping matched the ARP rule, which sends the ARP requests to ONOS.

```
mininet> sh ovs-ofctl -O OpenFlow13 dump-flows s2
 cookie=0x10000dc56d70b, duration=5.683s, table=0, n_packets=2, n_bytes=278, send_flow_rem priority=40000,dl_type=0x8942 actions=CONTROLLER:65535,clear_actions
 cookie=0x1000002bbd8d4, duration=5.683s, table=0, n_packets=2, n_bytes=278, send_flow_rem priority=40000,dl_type=0x88cc actions=CONTROLLER:65535,clear_actions
 cookie=0x10000c70edd85, duration=5.683s, table=0, n_packets=0, n_bytes=0, send_flow_rem priority=40000,arp actions=CONTROLLER:65535,clear_actions
 cookie=0x100002341485c, duration=5.683s, table=0, n_packets=0, n_bytes=0, send_flow_rem priority=5,ip actions=CONTROLLER:65535,clear_actions
```

Figure 66: Default rules on s2.

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4 h5
h2 -> h1 h3 h4 h5
h3 -> h1 h2 h4 h5
h4 -> h1 h2 h3 h5
h5 -> h1 h2 h3 h4
*** Results: 0% dropped (20/20 received)
```

Figure 67: `pingall` with ONOS.

```
mininet> sh ovs-ofctl -O OpenFlow13 dump-flows s2
 cookie=0x10000dc56d70b, duration=20.145s, table=0, n_packets=12, n_bytes=1668, send_flow_rem priority=40000,dl_type=0x8942 actions=CONTROLLER:65535,clear_actions
 cookie=0x1000002bbd8d4, duration=20.145s, table=0, n_packets=12, n_bytes=1668, send_flow_rem priority=40000,dl_type=0x88cc actions=CONTROLLER:65535,clear_actions
 cookie=0x10000c70edd85, duration=20.145s, table=0, n_packets=31, n_bytes=1302, send_flow_rem priority=40000,arp actions=CONTROLLER:65535,clear_actions
 cookie=0x100002341485c, duration=20.146s, table=0, n_packets=14, n_bytes=1372, send_flow_rem priority=5,ip actions=CONTROLLER:65535,clear_actions
 cookie=0x93000007e5a9e9, duration=9.537s, table=0, n_packets=1, n_bytes=98, send_flow_rem priority=10,in_port="s2-eth1",dl_src=00:00:00:00:00:02,dl_dst=00:00:00:00:00:01 actions=output:"s2-eth2"
 cookie=0x930000c33470bb, duration=9.513s, table=0, n_packets=1, n_bytes=98, send_flow_rem priority=10,in_port="s2-eth3",dl_src=00:00:00:00:00:04,dl_dst=00:00:00:00:00:01 actions=output:"s2-eth2"
 cookie=0x930000c204a428, duration=9.509s, table=0, n_packets=2, n_bytes=196, send_flow_rem priority=10,in_port="s2-eth3",dl_src=00:00:00:00:00:03,dl_dst=00:00:00:00:00:01 actions=output:"s2-eth2"
 cookie=0x930000038af3f48, duration=9.482s, table=0, n_packets=1, n_bytes=98, send_flow_rem priority=10,in_port="s2-eth2",dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:05 actions=output:"s2-eth3"
 cookie=0x930000089b416ca, duration=9.474s, table=0, n_packets=1, n_bytes=98, send_flow_rem priority=10,in_port="s2-eth3",dl_src=00:00:00:00:00:05,dl_dst=00:00:00:00:00:01 actions=output:"s2-eth2"
 cookie=0x930000b7402ef5, duration=9.470s, table=0, n_packets=1, n_bytes=98, send_flow_rem priority=10,in_port="s2-eth2",dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:04 actions=output:"s2-eth3"
 cookie=0x9300008417e3d0, duration=9.465s, table=0, n_packets=0, n_bytes=0, send_flow_rem priority=10,in_port="s2-eth2",dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:02 actions=output:"s2-eth1"
 cookie=0x930000664b01ce, duration=9.457s, table=0, n_packets=1, n_bytes=98, send_flow_rem priority=10,in_port="s2-eth2",dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:03 actions=output:"s2-eth3"
 cookie=0x9300007c402bce, duration=9.441s, table=0, n_packets=1, n_bytes=98, send_flow_rem priority=10,in_port="s2-eth1",dl_src=00:00:00:00:00:02,dl_dst=00:00:00:00:00:03 actions=output:"s2-eth3"
 cookie=0x9300009af305a1, duration=9.433s, table=0, n_packets=1, n_bytes=98, send_flow_rem priority=10,in_port="s2-eth3",dl_src=00:00:00:00:00:03,dl_dst=00:00:00:00:00:02 actions=output:"s2-eth1"
 cookie=0x930000f504a658, duration=9.427s, table=0, n_packets=1, n_bytes=98, send_flow_rem priority=10,in_port="s2-eth1",dl_src=00:00:00:00:00:02,dl_dst=00:00:00:00:00:04 actions=output:"s2-eth3"
 cookie=0x9300004cd81cfb, duration=9.417s, table=0, n_packets=1, n_bytes=98, send_flow_rem priority=10,in_port="s2-eth3",dl_src=00:00:00:00:00:04,dl_dst=00:00:00:00:00:02 actions=output:"s2-eth1"
 cookie=0x93000069f81275, duration=9.401s, table=0, n_packets=1, n_bytes=98, send_flow_rem priority=10,in_port="s2-eth1",dl_src=00:00:00:00:00:02,dl_dst=00:00:00:00:00:05 actions=output:"s2-eth3"
 cookie=0x930000bfb3bc91, duration=9.393s, table=0, n_packets=1, n_bytes=98, send_flow_rem priority=10,in_port="s2-eth3",dl_src=00:00:00:00:00:05,dl_dst=00:00:00:00:00:02 actions=output:"s2-eth1"
```

Figure 68: Rules on s2 after the ping.

```
mininet> sh ovs-ofctl -O OpenFlow13 dump-flows s1
 cookie=0x10000ea6f4b8e, duration=25.724s, table=0, n_packets=22, n_bytes=924, send_flow_rem priority=40000,arp actions=CONTROLLER:65535,clear_actions
 cookie=0x100007a585b6f, duration=25.724s, table=0, n_packets=8, n_bytes=1112, send_flow_rem priority=40000,dl_type=0x8942 actions=CONTROLLER:65535,clear_actions
 cookie=0x10000021b41dc, duration=25.724s, table=0, n_packets=8, n_bytes=784, send_flow_rem priority=5,ip actions=CONTROLLER:65535,clear_actions
 cookie=0x100009465555a, duration=25.724s, table=0, n_packets=8, n_bytes=1112, send_flow_rem priority=40000,dl_type=0x88cc actions=CONTROLLER:65535,clear_actions
 cookie=0x93000043a3c21c, duration=15.100s, table=0, n_packets=2, n_bytes=196, send_flow_rem priority=10,in_port="s1-eth2",dl_src=00:00:00:00:00:02,dl_dst=00:00:00:00:00:01 actions=output:"s1-eth1"
 cookie=0x9300008d053164, duration=15.089s, table=0, n_packets=1, n_bytes=98, send_flow_rem priority=10,in_port="s1-eth1",dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:03 actions=output:"s1-eth2"
 cookie=0x930000d134d2a8, duration=15.080s, table=0, n_packets=1, n_bytes=98, send_flow_rem priority=10,in_port="s1-eth2",dl_src=00:00:00:00:00:03,dl_dst=00:00:00:00:00:01 actions=output:"s1-eth1"
 cookie=0x930000721f2d0f, duration=15.075s, table=0, n_packets=1, n_bytes=98, send_flow_rem priority=10,in_port="s1-eth2",dl_src=00:00:00:00:00:04,dl_dst=00:00:00:00:00:01 actions=output:"s1-eth1"
 cookie=0x930000b8b18411, duration=15.047s, table=0, n_packets=1, n_bytes=98, send_flow_rem priority=10,in_port="s1-eth1",dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:05 actions=output:"s1-eth2"
 cookie=0x9300004e4458a2, duration=15.036s, table=0, n_packets=1, n_bytes=98, send_flow_rem priority=10,in_port="s1-eth2",dl_src=00:00:00:00:00:05,dl_dst=00:00:00:00:00:01 actions=output:"s1-eth1"
 cookie=0x93000f13c0a7d, duration=14.997s, table=0, n_packets=1, n_bytes=98, send_flow_rem priority=10,in_port="s1-eth1",dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:04 actions=output:"s1-eth2"
```

Figure 69: Rules on s1 before pinging 10.0.0.10.

```
mininet> sh ovs-ofctl -O OpenFlow13 dump-flows s1
 cookie=0x10000ea6f4b8e, duration=44.990s, table=0, n_packets=31, n_bytes=1302, send_flow_rem priority=40000,arp actions=CONTROLLER:65535,clear_actions
 cookie=0x100007a585b6f, duration=44.990s, table=0, n_packets=14, n_bytes=1946, send_flow_rem priority=40000,dl_type=0x8942 actions=CONTROLLER:65535,clear_actions
 cookie=0x10000021b41dc, duration=44.990s, table=0, n_packets=8, n_bytes=784, send_flow_rem priority=5,ip actions=CONTROLLER:65535,clear_actions
 cookie=0x100009465555a, duration=44.990s, table=0, n_packets=14, n_bytes=1946, send_flow_rem priority=40000,dl_type=0x88cc actions=CONTROLLER:65535,clear_actions
mininet>
```

Figure 70: Rules on s1 after pinging 10.0.0.10.

References

[1] Open Networking Foundation, *OpenFlow Switch Specification, Version 1.3.5 (Protocol version 0x04)*. Available: https://opennetworking.org/software-defined-standards/specifications/

[2] Ryu SDN Framework Community, *Ryu documentation*. Available: https://ryu.readthedocs.io/

[3] Ryu project team, *RYU SDN Framework* (Ryu Book), chapter "Switching Hub". Available: https://osrg.github.io/ryu-book/en/html/

[4] Mininet Team, *Introduction to Mininet*. Available: https://github.com/mininet/mininet/wiki/Introduction-to-Mininet

[5] CPqD, *ofsoftswitch13 – Dpctl Documentation*. Available: https://github.com/CPqD/ofsoftswitch13/wiki/Dpctl-Documentation

[6] Mininet Team, *Mininet Python API Reference Manual*. Available: http://mininet.org/api/

[7] Open vSwitch project, *ovs-ofctl(8) manual page*. Available: https://www.openvswitch.org/support/dist-docs/ovs-ofctl.8.txt

[8] Open vSwitch project, *ovs-vsctl(8) manual page*. Available: https://www.openvswitch.org/support/dist-docs/ovs-vsctl.8.txt

[9] OpenFlow Consortium, *OpenFlow Tutorial*. Available: http://archive.openflow.org/wk/index.php/OpenFlow_Tutorial

[10] B. Linkletter, *Using the OpenDaylight SDN Controller with the Mininet Network Emulator*, 2016. Available: https://www.brianlinkletter.com/2016/02/using-the-opendaylight-sdn-controller-with-the-mininet-network-emulator/

[11] Wikipedia, *Link Layer Discovery Protocol*. Available: https://en.wikipedia.org/wiki/Link_Layer_Discovery_Protocol

[12] jatj, *sdn_onos – INSTALL.md*. Available: https://github.com/jatj/sdn_onos/blob/master/INSTALL.md